

Decision Quotient: A Regime-Sensitive Complexity Theory of Exact Relevance Certification

Tristan Simas
McGill University
`tristan.simas@mail.mcgill.ca`

March 19, 2026

Abstract

Which coordinates of a decision problem can be hidden without changing the optimal action, and what is the computational cost of determining this? We analyze the complexity of this certification problem across three settings: static (counterexample exclusion), stochastic (conditioning and expectation), and sequential (temporal structure). In the static regime, minimum sufficiency is coNP-complete. In the stochastic regime, preservation and decisiveness separate: preservation is polynomial-time under explicit-state encoding, while decisiveness is PP-hard under succinct encoding with anchor and minimum variants in NP^{PP} . In the sequential regime, all queries are PSPACE-complete. We prove an encoding-sensitive contrast between explicit-state tractability and succinct-encoding hardness, identify six natural structural restrictions under which the problem becomes polynomial-time solvable, and derive an integrity-competence trilemma. A Lean 4 artifact mechanically verifies the optimizer-quotient universal property, main reductions, and finite decider core.

1 Introduction

Which coordinates of a decision problem can be hidden without changing the optimal action, and what is the computational cost of determining this? We study this question for a decision problem $\mathcal{D} = (A, S, U)$ with $S = X_1 \times \cdots \times X_n$. A coordinate set I is sufficient when agreement on I forces agreement on the optimal-action set:

$$s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s').$$

We classify how hard it is to certify this property across three settings: static, stochastic, and sequential. The problem is coNP-complete in the static regime, PP-complete in the stochastic regime, and PSPACE-complete in the sequential regime. Six natural structural restrictions yield polynomial-time certification.

This paper’s main claim is that once that canonical object is fixed, exact relevance certification becomes a regime-sensitive complexity problem. The structural target is the same throughout, but the cost of certifying it changes with the ambient model. In the static regime, certification

reduces to counterexample exclusion and collapses to relevance containment. In the stochastic regime, conditioning splits the exact question into preservation and decisiveness. In the sequential regime, temporal contingency lifts the same certification task to planning-style reasoning. The paper therefore has a simple backbone: the optimizer quotient supplies the canonical notion of exact decision-preserving compression, and the regime analyses classify when that structure can be certified exactly. (L: QT1, QT7.)

Informal interpretive guide. The optimizer quotient isolates the decision signal: it collapses exactly those state distinctions that do not change the optimal-action set. In that precise sense, it removes exactly decision noise and nothing decision-relevant. Relevant coordinates are points of leverage: changing them can change the decision. In the static regime, the resulting simplification is forced by structure rather than discovered by search. In the later consequence sections, the simplicity tax names the residual burden created when a system uses heuristic simplification in place of exact certification. (L: DN1-2.)

This is a certification problem rather than a forward-evaluation problem. Evaluating $U(a, s)$ or $\text{Opt}(s)$ on a fully specified state is one task; proving that a family of coordinates is irrelevant requires ruling out every counterexample pair of states. That universal structure is exactly what makes the optimizer quotient central rather than cosmetic: it records the maximal lossless collapse of the state space, and every exact decision-preserving abstraction must refine it. In the static regime, this structural picture sharpens further. A coordinate set is sufficient exactly when it exposes all relevant coordinates, so the minimum sufficient set is not discovered by a separate alternating search. It is the relevant-coordinate set itself. By contrast, ANCHOR-SUFFICIENCY retains a genuine existential choice because the anchor assignment must still satisfy a universal fiberwise condition.

Probability splits the certification problem. In stochastic decision problems, exact analysis separates into a preservation question and a decisiveness question. Preservation asks whether coarse information reproduces the full-information optimizer, so it is the direct stochastic analogue of exact quotient preservation. Decisiveness asks whether each observable fiber already determines a unique Bayes-optimal action, so it is a stronger decision-completeness demand. Formally, these are *stochastic sufficiency* and *stochastic decisiveness*. They are distinct, and the paper treats them separately on purpose. Preservation gives the bridge back to static sufficiency and, under full support, back to the same quotient structure as the static regime, while beyond full support the paper isolates support-sensitive bridge and obstruction lemmas rather than claiming a full general-distribution classification. Decisiveness yields the paper’s strongest succinct-encoding hardness classification. The stochastic row is therefore not one theorem about one predicate, but a split exact theory with two different complexity behaviors.

This is also what the complexity classification buys beyond nearby literatures on reducts, feature selection, or abstraction heuristics. Those traditions often ask whether some smaller representation is informative enough, predictive enough, or value-preserving enough. The present paper asks a sharper question: when can one certify exactly that hiding coordinates leaves the optimal-action correspondence unchanged? That exactness requirement is what later yields a no-general exact minimizer theorem, a regime-sensitive tractability boundary, and a direct reliability corollary about exact certifiers in the hard regime.

Connection to backdoor tractability in CSPs. Our coordinate sufficiency framework is structurally parallel to the backdoor framework for constraint satisfaction problems [2]. In their setting, a *backdoor* is a minimal variable set whose fixation collapses a CSP to a tractable class (e.g., one

admitting a majority polymorphism). In our setting, a *sufficient set* is a minimal coordinate set whose observation collapses a decision problem to a determined optimal-action set. Their result that PARTITION-MAJORITY-CSP is W[2]-hard when the tractable subset is unknown, but fixed-parameter tractable (FPT) when that subset is given, mirrors our result that identifying decision-relevant coordinates is coNP-complete and Σ_2^P -complete, while exploiting a known sufficient set is polynomial-time. Both frameworks formalize the same insight: discovering what matters is harder than exploiting it once known.

This distinction also explains why exactness matters. When a system replaces exact relevance certification with a heuristic simplification, the omitted relevance does not disappear; it is shifted into residual uncertainty that must be handled elsewhere in the system. We refer to that externalized burden as the *simplicity tax*. The results of this paper give that term formal content: exact simplification can be computationally expensive, but inexact simplification does not erase decision-relevant structure; it relocates it.

This systematic development reveals a regime-sensitive complexity landscape because the same structural target is being certified under richer models. Static certification is governed by counterexample exclusion. In the stochastic regime, preservation retains the quotient-preservation target while decisiveness introduces conditional-comparison hardness. In the sequential regime, omitted information can matter only through future contingencies, so exact certification becomes planning-hard. Read this way, the complexity matrix is not a list of unrelated classifications. It is the cost profile of certifying the exact decision-relevant distinctions encoded by the optimizer quotient.

Throughout, we work with finite coordinate-structured decision problems under the encoding conventions fixed in Section 2.4.

1.1 Contributions

The paper is best read as a theory of exact decision-relevant information organized around one canonical mathematical object: the optimizer quotient. The quotient gives the coarsest exact abstraction that preserves optimal-action distinctions, and this canonical status is theorem-level in the artifact. The paper then classifies how the cost of certifying that structure changes across static, stochastic, and sequential regimes, identifies tractable structural subcases, and supports the finite combinatorial core with a Lean artifact.

1. **Optimizer quotient as canonical structure.** For a finite decision problem, the optimizer quotient identifies states exactly when they induce the same optimal-action set. In **Set**, it is the coarsest exact decision-preserving abstraction, equivalently the coimage/image factorization of the optimizer map. This gives the paper a common structural object through which the later static, stochastic, and sequential results can be compared.
2. **Regime-sensitive certification landscape.** The paper studies how hard it is to certify exact preservation of decision-relevant structure across three regimes. In the static regime, sufficiency collapses to relevance containment, so MINIMUM-SUFFICIENT-SET is coNP-complete rather than the expected Σ_2^P search problem, while ANCHOR-SUFFICIENCY remains Σ_2^P -complete. This collapse is structurally nontrivial. In classical rough-set theory, general decision tables can admit multiple incomparable minimal reducts, which drives the minimum-reduct problem into combinatorial search territory. Under the exact optimal-action preservation predicate studied here, Proposition 2.9 proves there is exactly one minimal sufficient set, the relevant-coordinate set itself. This uniqueness is the structural reason for the collapse to coNP relevance containment. To our knowledge, this specific structural collapse, from Σ_2^P search to coNP driven by optimizer-quotient uniqueness, is a novel classification not captured

by general rough-set bounds. In the stochastic regime, exact analysis splits into preservation and decisiveness. Preservation is polynomial-time under explicit-state encoding, bridges back to static sufficiency and the optimizer quotient, and under full support yields inherited coNP and Σ_2^P classifications for the minimum and anchor variants. Decisiveness is polynomial-time in explicit state and PP-hard under succinct encoding, with the anchor and minimum variants PP-hard and in NP^{PP} at paper level. In the sequential regime, sufficiency, minimum, and anchor queries are PSPACE-complete.

3. **Tractability results.** The paper proves an explicit-state versus succinct-encoding contrast and identifies six natural structural restrictions under which exact certification becomes polynomial-time: bounded actions, separable utility, low tensor rank, tree structure, bounded treewidth, and coordinate symmetry. Each removes a distinct source of hardness.
4. **Consequences for exact certifiers.** In the hard exact regime, certifiers cannot in general be simultaneously sound, complete, and polynomial-budgeted. The same theory yields witness-checking lower bounds, approximation obstructions, and the practical consequence called the simplicity tax: when exact relevance is not certified, the unresolved burden is not removed but shifted elsewhere in the system.
5. **Mechanized finite-decision core.** A Lean 4 artifact mechanically checks the optimizer-quotient universal-property core, the main finite deciders, search procedures, reduction-correctness lemmas, witness/checking schemas, and bridge results (*L: QT1, QT7, DC91-99, OUI-2, OUI-12, EH1-10.*) The oracle-class placements are proved in the paper text, while the artifact independently verifies the finite combinatorial core on which those arguments rely. This places the mechanization in the emerging category of problem-specific certified reduction infrastructure.

1.2 Quantifier Structure

The complexity gap comes from the structure of the question. Insufficiency has short witnesses: two states that agree on the proposed coordinates but induce different optimal-action sets. Sufficiency, by contrast, asks for the absence of every such witness. So the object-level task is forward evaluation on a fully specified state, while the meta-level task is universal verification over counterexample pairs. This difference drives the coNP classification and explains why relevance identification is not just another forward-evaluation problem.

The static regime also contains the paper’s main structural simplification. Although minimum sufficiency can be written with an outer existential quantifier over coordinate sets, the existential layer collapses once sufficiency is characterized as containment of all relevant coordinates. This Σ_2^P -to-coNP collapse mirrors a related phenomenon in backdoor tractability for CSPs, where identifying a minimal backdoor to a tractable class is W[2]-hard [2], while exploiting a given backdoor is fixed-parameter tractable. The stochastic and sequential regimes do not simplify in the same way: the stochastic regime separates into preservation and decisiveness, with full-support inheritance on the preservation side and counting-style hardness on the decisiveness side, while temporal evolution introduces dependence on temporally induced optimizer structure.

1.3 Paper Structure

Section 2 introduces the formal setup and encoding conventions. Sections 3, 4, 5, and 6 develop the core theorem package. Sections 7 and 8 give the encoding-sensitive contrast and tractable subcases. Section 9 collects implications for exact simplification and reliability consequences for exact certifiers. Section 10 situates the work relative to formalized complexity and decision theory.

1.4 Artifact Availability

The proof artifact is archived at <https://doi.org/10.5281/zenodo.18998870>. The cited-content snapshot contains 22068 lines across 67 files with 0 occurrences of `sorry`.

2 Formal Setup

This section fixes the abstract vocabulary used throughout the paper. We work with finite decision problems whose state spaces factor into coordinates, and we isolate the minimal terminology needed to state the later complexity results. The same setup will support the paper’s three core regime classifications, the encoding-sensitive boundary, and the later consequence sections.

Object	Quantifier shape	Certifies	Role
SUFFICIENCY-CHECK	universal over state pairs	exact coordinate sufficiency	baseline certification predicate
MINIMUM-SUFFICIENT-SET	apparent $\exists I \forall (s, s')$ check	smallest sufficient support	collapses to relevance containment in the static regime
ANCHOR-SUFFICIENCY	$\exists$ anchor, then universal fiber check	exact preservation on one chosen fiber	retains the outer existential layer
Optimizer quotient	universal factorization property	coarsest decision-preserving abstraction	canonical structural object

Setup map. The four central objects differ mainly in quantifier shape: minimum sufficiency collapses in the static regime, while anchor sufficiency keeps its existential fiber choice.

2.1 Decision Problems with Coordinate Structure

Definition 2.1 (Decision Problem). A *decision problem with coordinate structure* is a tuple $\mathcal{D} = (A, X_1, \dots, X_n, U)$ where:

- A is a finite set of *actions* (alternatives)
- $X_1, \dots, X_n$ are finite *coordinate spaces*
- $S = X_1 \times \dots \times X_n$ is the *state space*
- $U : A \times S \rightarrow \mathbb{Q}$ is the *utility function*

Definition 2.2 (Projection). For state $s = (s_1, \dots, s_n) \in S$ and coordinate set $I \subseteq \{1, \dots, n\}$:

$$s_I := (s_i)_{i \in I}$$

is the *projection* of s onto coordinates in I .

Definition 2.3 (Optimizer Map). For state $s \in S$, the *optimal action set* is:

$$\text{Opt}(s) := \arg \max_{a \in A} U(a, s) = \{a \in A : U(a, s) = \max_{a' \in A} U(a', s)\}$$

2.2 Sufficiency and Relevance

Definition 2.4 (Sufficient Coordinate Set). A coordinate set $I \subseteq \{1, \dots, n\}$ is *sufficient* for decision problem $\mathcal{D}$ if:

$$\forall s, s' \in S : s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

Equivalently, the optimal action depends only on coordinates in I .

Proposition 2.5 (Empty-Set Sufficiency Equals Constant Decision Boundary). *The empty set $\emptyset$ is sufficient if and only if $\text{Opt}(s)$ is constant over the entire state space.*

(L: DP6.)

Proof. If $\emptyset$ is sufficient, then every two states agree on their empty projection, so sufficiency forces $\text{Opt}(s) = \text{Opt}(s')$ for all $s, s' \in S$. The converse is immediate. ■

Proposition 2.6 (Insufficiency Equals Counterexample Witness). *Coordinate set I is not sufficient if and only if there exist states $s, s' \in S$ such that $s_I = s'_I$ but $\text{Opt}(s) \neq \text{Opt}(s')$.*

(L: DP7.)

Proof. This is the direct negation of Definition 2.4. ■

Definition 2.7 (Minimal Sufficient Set). A sufficient set I is *minimal* if no proper subset $I' \subsetneq I$ is sufficient.

Definition 2.8 (Relevant Coordinate). Coordinate i is *relevant* if there exist states that differ only at coordinate i and induce different optimal-action sets:

$$i \text{ is relevant} \iff \exists s, s' \in S : \left(\forall j \neq i, s_j = s'_j \right) \wedge \text{Opt}(s) \neq \text{Opt}(s').$$

Proposition 2.9 (Minimal-Set/Relevance Equivalence). *For any minimal sufficient set I and any coordinate i ,*

$$i \in I \iff i \text{ is relevant}.$$

Hence every minimal sufficient set is exactly the relevant-coordinate set.

(L: DP1-2.)

Proof. If $i \in I$ were not relevant, then changing coordinate i while keeping all others fixed would never alter the optimal-action set, so removing i would preserve sufficiency, contradicting minimality. Conversely, every sufficient set must contain each relevant coordinate, because omitting a relevant coordinate leaves a witness pair with identical I -projection but different optimal-action sets. ■

Definition 2.10 (Exact Relevance Identifiability). For a decision problem $\mathcal{D}$ and candidate set I , we say that I is *exactly relevance-identifying* if

$$\forall i \in \{1, \dots, n\} : i \in I \iff i \text{ is relevant for } \mathcal{D}.$$

Definition 2.11 (Structural Rank). The *structural rank* of a decision problem is the number of relevant coordinates:

$$\text{srnk}(\mathcal{D}) := |\{i \in \{1, \dots, n\} : i \text{ is relevant}\}|.$$

2.3 Optimizer Quotient

Definition 2.12 (Decision Equivalence). States $s, s' \in S$ are *decision-equivalent* if they induce the same optimal-action set:

$$s \sim_{\text{Opt}} s' \iff \text{Opt}(s) = \text{Opt}(s').$$

Definition 2.13 (Decision Quotient). The *decision quotient* (equivalently, optimizer quotient) of $\mathcal{D}$ is the quotient object

$$Q_{\mathcal{D}} := S / \sim_{\text{Opt}}.$$

Its equivalence classes are exactly the maximal subsets of states on which the optimal-action set is constant.

The decision quotient is the canonical mathematical object associated with exact decision preservation. It quotients the state space by optimizer-equivalence and thereby isolates precisely the distinctions that any decision-preserving abstraction must respect.

Proposition 2.14 (Optimizer Quotient as Coimage/Image Factorization). *The decision quotient $Q_{\mathcal{D}}$ is canonically in bijection with $\text{range}(\text{Opt})$. Equivalently, in **Set**, it is the coimage/image factorization of the optimizer map.*

(L: [DQ6-7.](#))

Proof. Two states are identified by $\sim_{\text{Opt}}$ exactly when they have the same optimizer value. Hence quotient classes are in one-to-one correspondence with attained values of Opt . ■

Theorem 2.15 (Universal Property of the Optimizer Quotient). *Let $Q_{\mathcal{D}} = S / \sim_{\text{Opt}}$ be the decision quotient with quotient map $\pi : S \rightarrow Q_{\mathcal{D}}$. For any surjective abstraction $\phi : S \rightarrow T$ such that*

$$\phi(s) = \phi(s') \implies \text{Opt}(s) = \text{Opt}(s'),$$

there exists a unique map $\psi : T \rightarrow Q_{\mathcal{D}}$ such that

$$\pi = \psi \circ \phi.$$

(L: [QT1](#), [QT7.](#))

Proof. Because ϕ preserves optimizer values on its fibers, any two states identified by ϕ are decision-equivalent. So each fiber of ϕ is contained in a single $\sim_{\text{Opt}}$ -class.

For each $t \in T$, choose any $s \in S$ with $\phi(s) = t$ and define $\psi(t) := \pi(s)$. This is well defined because any two such choices lie in the same $\sim_{\text{Opt}}$ -class. By construction, $(\psi \circ \phi)(s) = \pi(s)$ for every $s \in S$, so $\pi = \psi \circ \phi$. Uniqueness follows from surjectivity of ϕ : if ψ' also satisfies $\pi = \psi' \circ \phi$, then every $t \in T$ has the form $t = \phi(s)$ and therefore

$$\psi(t) = \psi(\phi(s)) = \pi(s) = \psi'(\phi(s)) = \psi'(t).$$

Hence $\psi = \psi'$. ■

Remark 2.16 (Canonical Status). The optimizer quotient is therefore the coarsest abstraction of the state space that preserves optimal-action distinctions. Any surjective abstraction that is strictly coarser must erase some decision-relevant distinction.

(L: [QT7](#), [AB2.](#))

Remark 2.17 (Decision Noise). The quotient viewpoint identifies decision noise exactly. At the state level, decision noise is the kernel of the optimizer map: two states differ only by decision noise if and only if they determine the same class in $Q_{\mathcal{D}}$. At the coordinate level, a coordinate is pure decision noise if and only if varying it alone never changes the decision, equivalently if and only if it is irrelevant. In the finite probabilistic reading used later, this is also equivalent to conditional independence of the decision from that coordinate given the remaining coordinates.

(L: [DN1-2](#), [DN5](#).)

Proposition 2.18 (Sufficiency Characterization). *Coordinate set I is sufficient if and only if the optimizer map factors through the projection $\pi_I : S \rightarrow S_I$. Equivalently, there exists a function $\overline{\text{Opt}}_I : S_I \rightarrow \mathcal{P}(A)$ such that*

$$\text{Opt} = \overline{\text{Opt}}_I \circ \pi_I.$$

(L: [DQ42](#).)

Proof. If I is sufficient, define $\overline{\text{Opt}}_I(x)$ to be $\text{Opt}(s)$ for any state s with $s_I = x$; this is well defined exactly because sufficiency makes Opt constant on each projection fiber. Conversely, any such factorization forces states with equal I -projection to have equal optimal-action sets. ■

2.4 Computational Model and Input Encoding

The same decision question yields different computational problems under different input models. We therefore fix the encoding regime before stating any complexity classification.

Succinct encoding (primary for hardness). An instance is encoded by:

- a finite action set A given explicitly,
- coordinate domains $X_1, \dots, X_n$ given by their sizes in binary,
- a Boolean or arithmetic circuit C_U that on input (a, s) outputs $U(a, s)$.

The input length is

$$L = |A| + \sum_i \log |X_i| + |C_U|.$$

Unless stated otherwise, all class-theoretic hardness results in this paper are measured in L .

Explicit-state encoding. The utility function is given as a full table over $A \times S$. The input length is

$$L_{\text{exp}} = \Theta(|A||S|)$$

up to the bitlength of the utility entries. Polynomial-time claims stated in terms of $|S|$ use this explicit-state regime.

Query-access regime. The solver is given oracle access to $\text{Opt}(s)$ or to the utility values needed to reconstruct it. Complexity is then measured by query count, optionally paired with per-query evaluation cost. This regime separates access obstruction from full-table availability and from succinct circuit input length.

Remark 2.19 (Decision Questions vs Decision Problems). The predicate “is coordinate set I sufficient for $\mathcal{D}$?” is an encoding-independent mathematical question. A *decision problem* arises only after fixing how $\mathcal{D}$ and I are represented. Complexity classes are properties of these encoded problems, not of the abstract predicate in isolation.

Remark 2.20 (Later Stochastic Comparison). In the stochastic row, the preservation predicate compares the optimizer seen under coarse conditioning on I with the fully conditioned optimizer. Under full support, the later bridge package reconnects this stochastic notion to the same quotient picture as the static regime.

3 Static Regime Complexity

This section studies the static regime, where the input is a finite coordinate-structured decision problem under the succinct encoding of Section 2.4. This is the baseline certification setting: no stochastic averaging, no temporal dynamics, only the exact question of whether a candidate coordinate set preserves the optimizer map. The main contribution here is structural: sufficiency is exactly relevance containment. Once that characterization is in place, the complexity classification for MINIMUM-SUFFICIENT-SET follows immediately. The only genuinely higher-level static query is the anchor variant, where an existentially chosen fiber must satisfy a universal constancy condition.

3.1 Problem Definitions

Problem 3.1 (SUFFICIENCY-CHECK). **Input:** Decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ and coordinate set $I \subseteq \{1, \dots, n\}$.

Question: Is I sufficient for $\mathcal{D}$?

Problem 3.2 (MINIMUM-SUFFICIENT-SET). **Input:** Decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ and integer k .

Question: Does there exist a sufficient set I with $|I| \leq k$?

Problem 3.3 (ANCHOR-SUFFICIENCY). **Input:** Decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ and fixed coordinate set I .

Question: Does there exist an assignment α to I such that $\text{Opt}(s)$ is constant on the fiber $\{s : s_I = \alpha\}$?

3.2 Apparent Σ_2^P Structure

At first glance, MINIMUM-SUFFICIENT-SET seems to have the form

$$\exists I \forall s, s' \in S : s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s').$$

That syntax suggests a Σ_2^P classification. The algorithm suggested by this structure is: guess a coordinate set I , then universally verify that every agreeing state pair induces the same optimal-action set. On its face, static exact minimization therefore looks like a genuine existential–universal search problem.

3.3 Structural Characterization of Minimum Sufficiency

Proposition 2.9 removes the outer search layer: a set is sufficient exactly when it contains every relevant coordinate, so the minimum sufficient set is simply the relevant-coordinate set. This is the

central structural theorem of the static regime. Static exact minimization is therefore a relevance-counting problem, not a genuinely alternating search problem.

That collapse is specific to minimum sufficiency. ANCHOR-SUFFICIENCY still asks the algorithm to choose an assignment α and then certify a universal condition on the induced fiber, so its existential layer cannot be absorbed into a structural characterization of the instance.

3.4 Complexity Consequence: coNP-Completeness

The first two results show that exact certification is already harder than pointwise evaluation. A counterexample to sufficiency is short, but certifying that no such counterexample exists is a universal statement over state pairs. The collapse discussed above explains why the same coNP behavior governs both direct sufficiency checking and exact minimization.

Proof idea. Membership is immediate from short counterexamples: a “no” witness is just a pair of states that agree on I but induce different optimal-action sets. Hardness comes from reducing TAUTOLOGY to an empty-information instance whose optimizer map is constant exactly in the tautology case.

Theorem 3.4 (SUFFICIENCY-CHECK is coNP-complete). *SUFFICIENCY-CHECK is coNP-complete.*

(L: DQ40-41.)

Proof. For membership, Proposition 2.6 gives a polynomial witness (s, s') with $s_I = s'_I$ and $\text{Opt}(s) \neq \text{Opt}(s')$ whenever I is not sufficient, so the complement is in NP.

For hardness, reduce TAUTOLOGY to the empty-set instance. Given formula $\varphi(x_1, \dots, x_n)$, build a two-action decision problem where one action tracks φ and the other is a baseline action. Then $I = \emptyset$ is sufficient iff φ is a tautology, because empty-set sufficiency is exactly constancy of Opt (Proposition 2.5). The reduction is polynomial in formula size. ■

Theorem 3.5 (Static Structural Theorem and Complexity Consequence). *In the static regime, the apparent existential layer in MINIMUM-SUFFICIENT-SET collapses to relevance containment. Consequently, MINIMUM-SUFFICIENT-SET is coNP-complete.*

(L: DQ35-36.)

Proof idea. Once sufficiency is equivalent to containing all relevant coordinates, the question “is there a sufficient set of size at most k ?” becomes “are there at most k relevant coordinates?” The search component disappears, and the problem inherits the same coNP character as direct sufficiency checking.

Proof. Proposition 2.9 identifies the minimum sufficient set with the relevant-coordinate set. The question “ $|I^*| \leq k$?” therefore becomes the question whether at most k coordinates are relevant, which is a coNP check over relevance witnesses. CoNP-hardness follows from the $k = 0$ slice, which is exactly SUFFICIENCY-CHECK with $I = \emptyset$. ■

3.5 Sigma-2-P Completeness

Anchor sufficiency adds one more layer of choice that the previous collapse does not remove: the algorithm may select a fiber, but must then certify that the chosen fiber is uniformly decision-preserving. That existential-then-universal pattern is exactly what raises the complexity class.

Proof idea. Membership follows the visible quantifier pattern: guess an anchor assignment and then verify constancy on its fiber. Hardness is obtained by encoding the existential variables of an $\exists\forall$ formula into the anchor choice and the universal variables into the residual coordinates that must be checked.

Theorem 3.6 (ANCHOR-SUFFICIENCY is Sigma-2-P-complete). *ANCHOR-SUFFICIENCY is Σ_2^P -complete.*

(L: [DQ21-22](#).)

Proof. Membership follows directly from the quantifier pattern:

$$\exists\alpha \forall s \in S : (s_I = \alpha) \Rightarrow \text{Opt}(s) = \text{Opt}(s_\alpha).$$

For hardness, reduce from $\exists\forall$ -SAT: existential variables encode the anchor assignment, and universal variables range over the residual coordinates. Utility values are set so that fiberwise constancy holds exactly when the source formula is true. ■

3.6 Certificate-Size Lower Bound

The next theorem sharpens the static picture. It is not only that exact certification is hard in the complexity-class sense; even a checker restricted to the empty-set core may be forced to inspect exponentially many witness locations in the worst case.

Theorem 3.7 (Witness-Checking Duality). *For Boolean state spaces $S = \{0,1\}^n$, any sound checker for the empty-set sufficiency core must inspect at least 2^{n-1} witness pairs in the worst case.*

(L: [WD1-3](#).)

Proof. Empty-set sufficiency asks whether Opt is constant on all 2^n states. A sound refutation-complete checker must be able to separate constant maps from maps that differ on some hidden antipodal partition. An adversary can place the first disagreement in any of 2^{n-1} independent pair slots; if fewer than 2^{n-1} slots are inspected, two instances remain indistinguishable to the checker but have opposite truth values. Therefore at least 2^{n-1} pair checks are necessary. ■

Mechanization note. The TAUTOLOGY reduction stack, the coNP and Σ_2^P classifications, and the witness-budget lower-bound core are indexed by their inline Lean handles.

Explicit-state search upper bounds. Under the explicit-state step-counting model, static sufficiency and static anchor sufficiency are also wrapped as abstract P predicates on inputs carrying a certified state budget. The artifact proves these upper bounds via counted searches with quadratic and linear state-space dependence respectively, and it also packages the static sufficiency search as an explicit correctness-plus-step-bound witness and as part of the unified finite-search summary theorem. (L: [DC70-71](#), [DC76](#), [DC80-82](#).)

This completes the baseline regime. The next section keeps the same exact certification question but replaces pairwise counterexample exclusion by conditional expected-utility comparison, which is why the complexity lifts from the static coNP/ Σ_2^P picture to PP.

4 Stochastic Regime Complexity

We now add a distribution over states. Once utilities are compared through conditional expectations, the stochastic regime has two exact predicates rather than one. The first is a preservation predicate: does the optimizer seen after hiding coordinates agree with the optimizer seen when the full state is known? The second is a decisiveness predicate: does each observed fiber admit a unique Bayes-optimal action? Formally, these are stochastic sufficiency and stochastic decisiveness. Stochastic sufficiency gives the direct stochastic analogue of static sufficiency and the bridge back to the optimizer quotient; under full support, its minimum and anchor variants inherit the static coNP and Σ_2^P classifications. Stochastic decisiveness yields the strongest succinct-encoding complexity classification established in the paper.

4.1 Preservation and Decisiveness

In static decision problems, the exact question is whether hiding coordinates preserves the optimizer. In stochastic decision problems, preservation is still a well-defined exact question, but it is no longer the only one. One can also ask whether the retained signal is itself a complete decision interface, meaning that every observable fiber has a unique conditional optimum. That is the decisiveness predicate. The two predicates are not equivalent: a coordinate set may preserve the full-information optimizer without being decisive, and it may be decisive without preserving the full-information optimizer. Probability therefore produces two parallel exact questions rather than one uniform stochastic analogue.

4.2 Stochastic Decision Problems

Definition 4.1 (Stochastic Decision Problem). A stochastic decision problem is a tuple

$$\mathcal{D}_S = (A, X_1, \dots, X_n, U, P),$$

where $P \in \Delta(S)$ is a distribution on $S = X_1 \times \dots \times X_n$.

Definition 4.2 (Distribution-Conditional Sufficiency). For each admissible fiber value $\alpha \in \text{supp}(S_I)$, define the conditional optimal-action set

$$\text{Opt}_I^{\text{stoch}}(\alpha) := \arg \max_{a \in A} \mathbb{E}[U(a, S) \mid S_I = \alpha].$$

A coordinate set I is *stochastically sufficient* if conditioning on I preserves the fully conditioned optimizer:

$$\forall s \in S : \quad \text{Opt}_I^{\text{stoch}}(s_I) = \text{Opt}(s).$$

Here $S \sim P$ is the random state drawn from the instance distribution.

Intuition. The conditional optimizer $\text{Opt}_I^{\text{stoch}}(\alpha)$ aggregates expected utilities over the entire fiber $\{s : s_I = \alpha\}$ using the distribution P . By contrast, the full-information optimizer $\text{Opt}(s)$ conditions on the exact state s itself. Preservation asks whether this averaging over a fiber ever changes which action is optimal; decisiveness asks whether, after averaging, the optimal action on each fiber is uniquely determined.

Definition 4.3 (Stochastic Decisiveness). Using the same conditional optimizer map $\text{Opt}_I^{\text{stoch}}$, a coordinate set I is *stochastically decisive* if every admissible fiber has a uniquely determined conditional optimal action:

$$\forall \alpha \in \text{supp}(S_I) \exists a \in A : \quad \text{Opt}_I^{\text{stoch}}(\alpha) = \{a\}.$$

In the paper, we refer to this as *stochastic decisiveness*. In the current artifact, this is the stochastic predicate with the strongest succinct-encoding complexity package.

From witness exclusion to counting. Static sufficiency asks whether any counterexample pair exists. Stochastic sufficiency asks whether the conditional optimizer obtained from the retained coordinates still matches the fully conditioned optimizer, while stochastic decisiveness asks whether every admissible fiber has a uniquely determined conditional optimum. The first question is about ruling out disagreeing state pairs; the second and third are about conditional expected-utility comparisons on each fiber. That shift from witness exclusion to weighted counting comparison is exactly what brings stochastic hardness into the picture.

Example. Consider a stochastic decision problem with $S = \{s_1, s_2\}$, $A = \{a, b\}$, uniform distribution $P(s_1) = P(s_2) = 1/2$, and utilities $U(a, s_1) = 2$, $U(b, s_1) = 1$, $U(a, s_2) = 0$, $U(b, s_2) = 3$. For $I = \emptyset$: (i) Preservation: $\mathbb{E}[U(a, S)] = 1$, $\mathbb{E}[U(b, S)] = 2$, so $\text{Opt}_{\emptyset}^{\text{stoch}} = \{b\}$; but $\text{Opt}(s_1) = \{a\}$, so preservation fails. (ii) Decisiveness: The empty fiber itself has a unique conditional optimum $\{b\}$, so $I = \emptyset$ is decisive. This shows decisiveness is strictly stronger than preservation.

4.3 Stochastic Status at a Glance

Table 2. Regime classification summary for the stochastic row. Open entries mark explicit scope boundaries. Under full support, stochastic preservation is completely classified in the full-support base case.

Predicate/ query	What is proved here	Mechanized core	Open / not formalized
Stochastic sufficiency (preservation)	Polynomial-time under explicit-state encoding; bridges to static sufficiency and the optimizer quotient	Explicit-state decider; full-support bridge package	No succinct-encoding hardness claimed
Minimum / anchor preservation	Polynomial-time in explicit state; under full support, inherit coNP-completeness / Σ_2^P -completeness from the static regime	Full-support inheritance theorems; finite searches; obstruction/bridge lemmas	Open; conjecture coNP-hard
Stochastic decisiveness	Polynomial-time under explicit-state encoding; PP-hard under succinct encoding	Reduction core; finite witness/checking schemata	No oracle-machine formalization
Minimum / anchor decisiveness	PP-hard; in NPP^{PP} at paper level	Existential witness/checking schemata; bounded explicit-state searches	No oracle-class membership formalization

The rest of this section unpacks the four rows of this summary. The preservation side is complete in explicit state and under full-support inheritance, and now also has support-sensitive partial results

beyond full support. The decisiveness side carries the strongest succinct-encoding classification currently established in the paper.

Problem 4.4 (STOCHASTIC-SUFFICIENCY-CHECK). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I stochastically sufficient?

4.4 Stochastic Sufficiency

Theorem 4.5 (Stochastic Sufficiency is Decidable in Explicit State). *Under the explicit-state encoding, STOCHASTIC-SUFFICIENCY-CHECK is decidable in polynomial time.*

(L: [DC91-93](#), [OU12](#).)

Proof. For a fixed information set I , the verifier scans the finite state space and checks whether the conditional fiber optimizer $\text{Opt}_I^{\text{stoch}}(s_I)$ agrees with the fully conditioned optimizer $\text{Opt}(s)$ at every state s . The artifact contains a counted explicit-state search witnessing exactly this predicate together with a linear-in- $|S|$ step bound (L: [DC91-93](#).) Hence stochastic sufficiency is polynomial-time decidable in the explicit-state model. ■

Conjecture 4.1 (Support-Sensitive Preservation Complexity). Under the succinct encoding of Section 2.4, STOCHASTIC-PRESERVATION-CHECK for general distributions (without the full-support assumption) is coNP-hard.

The structured justification is anchored to existing theorems and mechanized lemmas. First, what is known: under full support, preservation inherits the static coNP classification via Proposition 4.7 and Theorem 4.11; the explicit-state base query is polynomial for all distributions by Theorem 4.5; and decisiveness under the same encoding is PP-hard by Theorem 4.19.

Second, the structural obstacle: the support-sensitive obstruction lemmas in the artifact (Proposition 4.14) isolate the exact mechanism. Preservation quantifies over all states while stochastic conditioning weights by probability. Zero-probability states can therefore witness static violations that the distribution never observes; full-support transfer fails exactly when such states are permitted. This is a structural obstruction, and the mechanized artifact isolates the finite combinatorial core any reduction must respect (L: [OU12](#), [DC96](#), [DC98](#)).

Third, why coNP rather than PP: preservation is syntactically universal, because it asks whether the coarse optimizer matches the full optimizer at every state, so its quantifier shape matches the static coNP classification. Decisiveness is PP-hard because it requires majority-style conditional expected-utility comparisons. Preservation compares optimizer sets rather than aggregating utilities, so the hardness source is counterexample exclusion (coNP) rather than majority counting (PP); support-sensitivity complicates reduction engineering but not the high-level quantifier structure.

Reduction intuition. A candidate hardness reduction would encode static counterexample pairs into zero-probability witness states of the distribution. Concretely, from a TAUTOLOGY instance φ , one could construct a succinct stochastic decision problem where the distribution assigns zero mass to a carefully chosen set of states that encode the φ -violating assignments. Preservation must check the universal condition over ALL states, including these zero-probability witnesses, so a coNP-style verification can detect the hidden TAUTOLOGY counterexample even though the distribution never observes it. This intuition aligns with the support-sensitive obstruction lemmas (Proposition 4.14) but requires overcoming the technical challenge of succinctly encoding witness-state support.

Open Problem 4.2. Determine the exact succinct-encoding complexity of STOCHASTIC-MINIMUM-PRESERVATION and STOCHASTIC-ANCHOR-PRESERVATION for general distributions. Under Conjecture 4.1, these inherit coNP-completeness and Σ_2^P -completeness respectively, matching the static regime; resolving the conjecture would complete the final open cell in the regime matrix (Table 2).

Proposition 4.6 (Stochastic Sufficiency Implies Static Sufficiency). *If I is stochastically sufficient, then I is sufficient for the underlying static decision problem.*

(L: DC94.)

Proof. If the coarse conditional optimizer agrees with the fully conditioned optimizer at every state, then any two states that agree on I also have equal full-information optimal-action sets. So the original static optimizer already factors through the projection to I . ■

Proposition 4.7 (Full-Support Equivalence). *Assume the distribution P has full support and the action set is nonempty. Then I is statically sufficient if and only if it is stochastically sufficient.*

(L: DC95-96.)

Proof. The forward direction uses full support to show that when the static optimizer is constant on each I -fiber, conditional averaging cannot change the optimizer on that fiber. The reverse direction is Proposition 4.6. Both directions are mechanized. ■

Proposition 4.8 (Quotient Equivalence under Full Support). *Under the hypotheses of Proposition 4.7, the stochastic fiber quotient induced by I coincides with the original decision quotient.*

(L: DC97-99.)

Proof. Under stochastic sufficiency, the conditional fiber optimizer and the fully conditioned optimizer agree statewise. Hence the induced stochastic equivalence relation matches the original decision-equivalence relation, so the quotient setoids coincide. Under full support, Proposition 4.7 supplies the needed preservation hypothesis. ■

Problem 4.9 (STOCHASTIC-MINIMUM-PRESERVATION). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $k \in \mathbb{N}$.

Question: Does there exist a stochastically sufficient coordinate set I with $|I| \leq k$?

Problem 4.10 (STOCHASTIC-ANCHOR-PRESERVATION). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $I \subseteq \{1, \dots, n\}$.

Question: Does there exist an anchor state s_0 such that for every state s agreeing with s_0 on I , one has $\text{Opt}_I^{\text{stoch}}(s_I) = \text{Opt}(s)$?

Theorem 4.11 (Full-Support Inheritance for Preservation Variants). *Assume the distribution P has full support and the action set is nonempty. Then:*

1. STOCHASTIC-MINIMUM-PRESERVATION is equivalent to the underlying static MINIMUM-SUFFICIENT-SET query.
2. For each fixed I , STOCHASTIC-ANCHOR-PRESERVATION is equivalent to the underlying static ANCHOR-SUFFICIENCY query on I .

Consequently, under full support, stochastic minimum preservation is coNP-complete and stochastic anchor preservation is Σ_2^P -complete.

Proof. For minimum preservation, Proposition 4.7 applies pointwise to each candidate coordinate set I , so the existential minimization problem is unchanged under full support. For anchor preservation, if one anchor fiber preserves the full-information optimizer, then the optimizer is already constant on that fiber, giving static anchor sufficiency. Conversely, if the static optimizer is constant on one I -fiber, then under full support the same conditional-expectation argument used in Proposition 4.7 shows that the coarse conditional optimizer agrees with the full-information optimizer throughout that fiber. The complexity claims therefore inherit directly from Theorems 3.5 and 3.6. ■

Proposition 4.12 (Explicit Finite Search for Preservation Variants). *For finite instances, the artifact contains counted exhaustive-search procedures for stochastic minimum preservation and stochastic anchor preservation. Their certified step bounds are $O(2^n)$ and $O(|S|)$, respectively.*

Proof. The minimum-preservation search scans the subset lattice and invokes the explicit-state preservation checker on each candidate set. The anchor-preservation search scans candidate anchor states and checks preservation on the induced fiber. Both procedures are packaged with correctness theorems and explicit step bounds in the artifact. ■

Theorem 4.13 (Explicit-State Tractability for Preservation Variants). *Under the explicit-state encoding, STOCHASTIC-MINIMUM-PRESERVATION and STOCHASTIC-ANCHOR-PRESERVATION are both decidable in polynomial time.*

Proof. For minimum preservation, the explicit-state search enumerates coordinate subsets and invokes the explicit preservation checker on each candidate set; the resulting counted procedure is polynomial in the explicit input size for fixed state and coordinate budgets. For anchor preservation, the explicit-state search enumerates anchor states and checks preservation on the induced fiber, yielding a polynomial-time verifier. Both wrappers are mechanized in the artifact. ■

Proposition 4.14 (General-Distribution Obstructions and Support-Sensitive Bridge). *Let I be a coordinate set.*

1. *If I is stochastically sufficient, then I contains every relevant coordinate of the underlying static decision problem.*
2. *Consequently, in the Boolean-cube setting, if STOCHASTIC-MINIMUM-PRESERVATION has a yes-instance with budget k , then the static relevant-coordinate set has cardinality at most k .*
3. *If the underlying static decision problem is sufficient on I and every I -fiber contains at least one positive-probability state, then I is stochastically sufficient.*
4. *If the underlying static decision problem is anchor-sufficient on I and the anchor fiber contains a positive-probability state, then I is stochastically anchor-preserving.*

Proof. The first statement composes the unconditional bridge from stochastic preservation to static sufficiency with the static theorem that every sufficient set contains all relevant coordinates. The second is the resulting cardinality bound for any witness set in the minimum query. For the positive-support statements, the mechanized proof refines the full-support argument: it uses positivity only on one representative state per relevant fiber, together with fiberwise invariance of the conditional optimizer, to propagate static sufficiency or anchor sufficiency into stochastic preservation on the whole fiber. ■

What this paper establishes for preservation. For stochastic sufficiency in the preservation sense, this paper proves explicit-state tractability for the base, minimum, and anchor queries (Theorem 4.5, Proposition 4.12, and Theorem 4.13), static-to-stochastic bridge theorems (Propositions 4.6 and 4.7), quotient equivalence under full support (Proposition 4.8), and full-support inheritance for the minimum and anchor variants (Theorem 4.11). Beyond full support, the paper now also isolates unconditional obstruction lemmas and positive-support bridge results (Proposition 4.14). This yields a complete classification for stochastic preservation under full support, the full-support base case in which every state has nonzero probability, together with support-sensitive partial theory beyond that base case. The general-distribution case still introduces technical complications from zero-probability fibers, and the paper does not claim a full general-distribution classification for the minimum and anchor preservation variants or a succinct-encoding hardness classification for the preservation predicate itself. The PP-hardness results in the stochastic row concern decisiveness and its anchor/minimum variants, not preservation itself.

Problem 4.15 (STOCHASTIC-DECISIVENESS-CHECK). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I stochastically decisive?

Naming convention. From this point onward, the historical names “stochastic anchor sufficiency” and “stochastic minimum sufficiency” refer to the decisiveness family, not the preservation family introduced above.

Definition 4.16 (Stochastic Anchor Sufficiency). A coordinate set I is *stochastically anchor-sufficient* if there exists an admissible fiber value $\alpha \in \text{supp}(S_I)$ and an action $a \in A$ such that

$$\arg \max_{a' \in A} \mathbb{E}[U(a', S) \mid S_I = \alpha] = \{a\}.$$

Equivalently, some I -fiber has a unique conditional optimal action.

Problem 4.17 (STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I stochastically anchor-sufficient?

Problem 4.18 (STOCHASTIC-MINIMUM-SUFFICIENT-SET). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $k \in \mathbb{N}$.

Question: Does there exist a stochastically decisive coordinate set I with $|I| \leq k$?

4.5 Encoding-Sensitive Complexity of Stochastic Decisiveness

The stochastic regime preserves the same underlying certification template, namely whether the retained coordinates determine the optimizer, but asks it after conditioning and expectation have been introduced. The current complexity package concerns stochastic decisiveness because it admits a clean mechanized reduction core and directly supports the existential anchor/minimum queries. For the empty-information case, one is already comparing aggregate mass of satisfying versus non-satisfying states. More generally, stochastic decisiveness asks whether one action strictly dominates all competitors in conditional expected utility on each admissible fiber.

Theorem 4.19 (Stochastic Decisiveness is Encoding-Sensitive). *For the stochastic decisiveness predicate above, STOCHASTIC-DECISIVENESS-CHECK is decidable in polynomial time under the explicit-state encoding. Under the succinct encoding it is PP-hard. The accompanying upper-bound argument is proved in the paper via an explicit bad-fiber witness characterization whose finite witness/checking core is mechanized in the artifact.*

(L: DC45, DC62, DC72, OU8-11.)

Proof. For the explicit-state encoding, the number of admissible fibers is at most $|S|$, so one can enumerate the fibers and, for each fiber value α , compute the conditional expected utilities of all actions directly from the table. Checking whether exactly one action attains the maximum on each fiber is therefore polynomial in the explicit input size.

For succinct-encoding hardness, reduce MAJSAT by the three-action gadget used in the mechanized reduction. Given Boolean formula φ , take $S = \{0,1\}^n$, uniform P , actions $\{\text{accept}, \text{hold}_L, \text{hold}_R\}$, and utilities

$$U(\text{accept}, s) = \mathbf{1}[\varphi(s)], \quad U(\text{hold}_L, s) = U(\text{hold}_R, s) = \frac{1}{2} - 2^{-(n+1)},$$

with $I = \emptyset$. There is only one admissible fiber, so stochastic decisiveness asks whether the prior-optimal action is unique. By construction, this happens exactly when $\mathbb{E}[\varphi] \geq 1/2$, i.e., in the MAJSAT case. Thus MAJSAT many-one reduces to stochastic decisiveness.

For the upper bound, failure of decisiveness is witnessed by one bad state whose observed fiber does not have a singleton conditional optimum. The current artifact packages exactly this no-witness characterization together with the corresponding existential witness schema, and the two directions are bundled in a summary theorem (L: OU8-11.) These mechanized witness/checking packages verify the finite combinatorial core used by the paper's oracle-class arguments. Concretely, the artifact verifies the existential-universal quantifier structure of the predicate, the witness-checking schema that guesses an anchor and verifies conditional-uniqueness using PP comparisons, and the finite-step-counted search wrappers (handles OU1 to OU2, OU6 to OU7) that bound the nondeterministic guessing phase. The standard oracle-machine reduction proving NP^{PP} membership is argued in the paper text and not mechanized. The corresponding coNP^{PP} -style and NP^{PP} -style memberships are then proved in the paper text by standard complexity-theoretic reasoning. ■

Proposition 4.20 (Stochastic Decisiveness Yields Stochastic Anchor Sufficiency). *If I is stochastically decisive, then I is stochastically anchor-sufficient.*

(L: DC19.)

Proof. Choose any admissible fiber value α . By stochastic decisiveness, there exists an action a such that

$$\text{Opt}_I^{\text{stoch}}(\alpha) = \{a\}.$$

That fiber therefore witnesses stochastic anchor sufficiency. ■

Theorem 4.21 (Stochastic Anchor Sufficiency is PP-hard). *STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK is PP-hard.*

(L: DC40-42.)

Proof. Reduce MAJSAT. Given Boolean formula φ on $n \geq 1$ variables, take $S = \{0,1\}^n$, uniform P , $I = \emptyset$, and three actions $\{\text{accept}, \text{hold}_L, \text{hold}_R\}$. Set

$$U(\text{accept}, s) = \mathbf{1}[\varphi(s)], \quad U(\text{hold}_L, s) = U(\text{hold}_R, s) = \frac{1}{2} - 2^{-(n+1)}.$$

If at least half of all assignments satisfy φ , then accept is the unique optimal action on the empty-information fiber, so the instance is anchor-sufficient. If fewer than half satisfy φ , then hold_L and hold_R tie above accept, so no singleton anchor optimum exists. Thus MAJSAT many-one reduces to the stochastic anchor query. ■

Theorem 4.22 (Stochastic Minimum-Sufficient-Set is PP-hard). *STOCHASTIC-MINIMUM-SUFFICIENT-SET is PP-hard.*

(L: DC45, DC47.)

Proof. Reduce MAJSAT to the $k = 0$ slice. In the same three-action gadget used above, there exists a stochastically decisive set of size at most 0 iff the empty set itself is stochastically decisive. This empty-set equivalence for the gadget is certified in the artifact, so MAJSAT many-one reduces to STOCHASTIC-MINIMUM-SUFFICIENT-SET. ■

Proposition 4.23 (Explicit Finite Search for Stochastic Queries). *For finite instances, the artifact contains counted exhaustive-search procedures for stochastic sufficiency, stochastic decisiveness, stochastic anchor sufficiency, and stochastic minimum sufficiency. Their certified step bounds are respectively $O(|S|)$, $O(|S|)$, $O(|S||A|)$, and $O(2^n)$.*

(L: DC62-65, DC72-73, DC76, DC91-93.)

Proof. The stochastic-sufficiency search scans for a state where the coarse conditional optimizer disagrees with the fully conditioned optimizer; the decisiveness search scans for a violating state whose fiber-optimal set is non-singleton; the anchor search scans over candidate anchor-state/action pairs; and the minimum query scans the subset lattice. Each procedure admits a counted boolean search formulation with both a correctness theorem and an explicit step bound. ■

Theorem 4.24 (Stochastic Anchor and Minimum Queries Lie in NP^{PP}). *Under the present encoding, STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK and STOCHASTIC-MINIMUM-SUFFICIENT-SET are both in NP^{PP} .*

(L: OU1-2, OU6-7, PA3, EH1-5.)

Proof. For anchor sufficiency, the mechanized collapse

$$\text{STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK}(P, I) \iff \exists s_0 \exists a (\text{fiberOpt}(P, I, s_0) = \{a\})$$

is already available in the artifact PA3. A nondeterministic polynomial-time machine may therefore guess (s_0, a) and use a PP oracle to verify that a is the unique conditional optimum on the anchor fiber. Concretely, uniqueness can be checked by comparing the conditional expected utility of a against every competing action on that fiber, which is exactly the same majority-style conditional comparison used in the decisiveness verification.

For the minimum query, the machine guesses a coordinate set I with $|I| \leq k$ and then asks whether I is stochastically decisive. The same witness/checking template used for the anchor case yields the required PP-style verifier for the guessed set, so the minimum query is also in NP^{PP} . ■

Theorem 4.24 completes the complexity picture for stochastic decisiveness: decisiveness is polynomial-time in the explicit-state model and PP-hard in the succinct model, while the anchor and minimum decisiveness queries are PP-hard and lie in NP^{PP} . Those oracle-class memberships are proved in the paper text. The artifact independently verifies the finite core behind those proofs: the bad-fiber counter-witness schema for decisiveness, the anchor-collapse equivalence, the generic existential-witness schema, bounded-witness recoveries for the explicit-state stochastic anchor and minimum searches, and existential-majority hardness/completeness packages for the anchor and decisiveness query families (L: OU8-11, EH2-10.)

Scope note. For stochastic preservation, this paper establishes explicit-state tractability for the preservation family, full-support inheritance, quotient equivalence, and support-sensitive obstruction/bridge lemmas beyond full support. The general-distribution minimum and anchor variants still remain open as full classification problems and are not claimed. For stochastic decisiveness, the paper establishes PP-hardness and NP^{PP} membership under the stated encoding. The open entries in Table 2 are explicit scope boundaries rather than unresolved subcases.

The remaining gap is not a bookkeeping omission but a real structural issue. Stochastic decisiveness is a pure verification problem: once I is fixed, one must check a family of majority-style conditional comparisons, which yields the stated encoding-sensitive complexity. By contrast, stochastic anchor and minimum sufficiency add an outer existential choice, namely to guess an anchor witness or a candidate coordinate set, before performing the same conditional verification. The current collapse theorem PA3 shows that anchor sufficiency reduces to existence of a fiber with a singleton conditional optimum, and the new existential-majority reduction shows that this extra witness layer is not vacuous. So the open question is genuinely whether that existential layer can be absorbed into PP under the present encoding, not whether the paper has simply stopped one step short of an obvious closure argument.

For that reason, the paper’s claim is intentionally calibrated. It does not present the stochastic row as one undifferentiated theorem about one predicate. Instead, it treats preservation and decisiveness as parallel exact questions. Preservation gives the direct stochastic analogue of the static question and the bridge back to the optimizer quotient. Decisiveness yields the strongest succinct-encoding complexity classification established in the paper. The split is part of the proved theory, not a temporary patch.

Relevance boundary. The stochastic decisiveness query is fiber-indexed: fixing a candidate information set I induces a derived decision problem whose optimizer is the conditional fiber optimizer. In the current formalization this induced problem is always I -sufficient (L: DC57), but its relevance notion is therefore also indexed by I . For that reason, the paper does not assert a single global relevance-set characterization for stochastic minimum sufficiency analogous to the static and sequential regimes.

4.6 Tractable Subcases

Proposition 4.25 (Tractable Stochastic Cases). *Stochastic sufficiency is polynomial-time solvable under each of the following restrictions:*

1. *product distributions $P = \bigotimes_i P_i$,*
2. *bounded support $|\text{supp}(P)| \leq k$,*
3. *structured families where conditional expectations are computable in polynomial time (e.g. log-concave models with oracle access).*

(L: DQ13, DQ15.)

Proof. In each case, conditional expected utilities can either be computed exactly or reduced to a polynomial number of marginal comparisons. ■

4.7 Bridge from Static

Distributional conditioning can genuinely destroy or preserve stochastic sufficiency depending on the structure of the input model.

Proposition 4.26 (Static Sufficiency Does Not Imply Stochastic Sufficiency). *There exist instances where I is sufficient in the static sense but not stochastically sufficient once a distribution P is fixed.*

(L: DQ58.)

Proof. Choose utilities where optimizer ties are harmless pointwise but are broken in expectation after conditioning. Then static fibers preserve Opt, while the conditional optimizer obtained from I fails to match the fully conditioned optimizer. ■

Proposition 4.27 (Static Singleton Sufficiency Transfers to Stochastic Sufficiency). *Suppose that on every I -fiber the static optimal-action set is a singleton, and that this singleton is constant on that fiber. Then I is stochastically sufficient for every distribution P .*

(L: DQ15-16.)

Proof. Fix an admissible fiber value α and let $\{a_\alpha\}$ be the common pointwise optimal-action set on that fiber. For every competing action $b \neq a_\alpha$ and every state s with $s_I = \alpha$, one has $U(a_\alpha, s) > U(b, s)$. Taking conditional expectations over the fiber preserves this strict inequality, so

$$\text{Opt}_I^{\text{stoch}}(\alpha) = \{a_\alpha\}.$$

Since this holds for every admissible fiber, the conditional optimizer on each fiber agrees with the fully conditioned optimizer at every state in that fiber. Hence I is stochastically sufficient. ■

The stochastic regime therefore gives the middle layer of the paper’s hierarchy: conditioning already raises exact certification beyond the static setting by turning exact certification into a counting comparison problem. Adding time will lift us further still: the next section shows how temporal contingency presents, alongside stochastic sufficiency, a decisiveness complexity family and then pushes the same underlying question from PP-style comparisons to PSPACE-level temporal certification.

5 Sequential Regime Complexity

We now move to sequential settings with transitions and observations. Here the same certification question is asked for a sequential decision model carrying transitions, observations, and a planning horizon. In the current formalization, the certified predicate is state-based: after packaging the sequential instance as its induced one-step decision map, can one suppress some coordinates without changing the resulting optimal-action set? This is where the complexity reaches PSPACE. The structural reason is that hidden information can matter later even when it appears irrelevant now. Certification must therefore account for temporally unfolding contingencies rather than compare only one-shot optimizers or conditional expectations. This complexity jump is consonant with the exact abstraction literature for temporally structured decision models: once hidden information can affect future contingent choices, exact aggregation and exact relevance certification both require reasoning about temporally mediated distinctions rather than one-shot optimizer comparisons. In succinct models, that temporal contingency supports the same kind of alternation that drives PSPACE hardness in planning and quantified-logic reductions.

5.1 Sequential Decision Problems

Definition 5.1 (Sequential Decision Problem). A sequential decision problem is a tuple

$$\mathcal{D}_{\text{seq}} = (A, X_1, \dots, X_n, U, T, O),$$

with state space $S = X_1 \times \dots \times X_n$, transition kernel $T : A \times S \rightarrow \Delta(S)$, and observation model $O : S \rightarrow \Delta(\Omega)$.

Definition 5.2 (Sequential Sufficiency). For a sequential instance $\mathcal{D}_{\text{seq}}$, let $\text{Opt}_{\text{seq}}(s)$ denote the optimal-action set of its induced decision map on state s . A coordinate set I is *sequentially sufficient* if

$$\forall s, s' \in S : s_I = s'_I \implies \text{Opt}_{\text{seq}}(s) = \text{Opt}_{\text{seq}}(s').$$

This is the sequential state-based certification predicate used throughout the paper and in the artifact.

Two interpretations. There are two closely related readings of this definition. Formally, the sequential row studies sufficiency for the optimizer induced by the sequential model. Its intended interpretation is policy-level: hidden information may be irrelevant at time zero and still become relevant later because it changes future contingent choices.

One can also read the same construction heuristically as a coordinate-restricted POMDP abstraction question. A full controller conditions on evolving observations and therefore on induced beliefs over latent states. Hiding coordinates changes the informational interface through which future decisions are made. That heuristic is useful for intuition and for understanding the TQBF gadget, but the formal theorems in this paper are stated for the state-based predicate above rather than for a fully developed history-dependent policy class.

Problem 5.3 (SEQUENTIAL-SUFFICIENCY-CHECK). **Input:** $\mathcal{D}_{\text{seq}}$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I sequentially sufficient?

Definition 5.4 (Sequential Anchor Sufficiency). A coordinate set I is *sequentially anchor-sufficient* if there exists an anchor state whose I -agreement class preserves the optimal action set throughout that class.

Problem 5.5 (SEQUENTIAL-ANCHOR-SUFFICIENCY-CHECK). **Input:** $\mathcal{D}_{\text{seq}}$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I sequentially anchor-sufficient?

Problem 5.6 (Sequential Minimum Query). **Input:** $\mathcal{D}_{\text{seq}}$ and $k \in \mathbb{N}$.

Question: Does there exist a sequentially sufficient coordinate set I with $|I| \leq k$?

Proposition 5.7 (Sequential Minimality Equals Relevance). *For any minimal sequentially sufficient set I , a coordinate lies in I if and only if it is relevant for the underlying decision boundary.*

(L: DC49-50.)

Proof. In the current formalization, sequential sufficiency is defined as sufficiency of the underlying one-step optimizer map on the state space. The static minimal-set/relevance equivalence therefore transports directly to the sequential setting. ■

5.2 PSPACE-Completeness

The sequential regime is the strongest of the three because hidden information can matter not only for the current optimizer but for the future evolution of the decision problem. Static certification asks whether bad state pairs exist; stochastic certification asks whether conditional averages cross a threshold; sequential certification asks whether omitted coordinates can change the induced optimizer after temporal structure is taken into account. That added temporal contingency is exactly what the TQBF reduction exploits.

The reduction follows the same structural pattern as quantified formulas and temporally structured control problems elsewhere in the planning literature. Existential quantifier blocks correspond to controller-side choices, universal blocks correspond to adversarial or environment-controlled resolutions, and the truth value of the quantified formula is encoded into the induced optimizer of the reduced sequential instance. A hidden coordinate can look locally irrelevant after one transition yet still encode which branch of the quantified game has been entered. The reduction exposes the source of PSPACE hardness: exact relevance certification must reason about information whose effect is mediated by temporal contingency rather than by a single one-shot comparison.

Theorem 5.8 (Sequential Sufficiency is PSPACE-complete). *SEQUENTIAL-SUFFICIENCY-CHECK is PSPACE-complete.*

(L: DC46, DC66, DC74.)

Proof. For membership, the verifier ranges over state pairs or candidate witnesses while evaluating the induced sequential optimizer of the input instance. Under the present finite-horizon encoding, that induced optimizer can be evaluated using only polynomial space in the succinct input description of the reduction gadgets, and the outer search can likewise be carried out in polynomial space. This is the verifier pattern abstracted by the explicit finite-search layer formalized in the artifact.

For hardness, reduce TQBF. Encode alternating quantifiers into the reduced sequential instance so that the induced optimizer is state-independent exactly when the quantified instance is true. Then empty-set sequential sufficiency holds exactly when the source TQBF instance is true. ■

The proof can be read operationally as follows. A sequential certificate must rule out the possibility that some hidden bit becomes decision-relevant only after several temporally mediated updates. That means the verifier cannot stop at a single static comparison; it has to account for contingent future structure in the induced optimizer. TQBF is therefore the correct reduction target: the same alternation between “there exists a choice” and “for every resolution” is already built into the reduced sequential instance.

Proposition 5.9 (Sequential Sufficiency Refines to Sequential Anchor Sufficiency). *If I is sequentially sufficient, then I is sequentially anchor-sufficient.*

(L: DC23.)

Proof. If optimal-action sets are preserved for every pair of states agreeing on I , then fixing any anchor state immediately yields preservation across its entire I -agreement class. ■

Theorem 5.10 (Sequential Anchor Query is PSPACE-complete). *The sequential anchor query is PSPACE-complete.*

(L: DC28-29, DC44.)

Proof. Membership is in PSPACE. By definition, the query asks whether there exists an anchor state s_0 such that every state agreeing with s_0 on I preserves the same optimal-action set. A nondeterministic polynomial-space procedure can guess s_0 and then scan candidate states one at a time, verifying the agreement condition and comparing the corresponding optimal-action sets. Since $\text{NPSpace} = \text{PSPACE}$, the query lies in PSPACE.

Reuse the TQBF reduction for sequential sufficiency with $I = \emptyset$. At empty information, sequential anchor sufficiency is equivalent to global preservation of the optimal-action set, so the same construction yields

$$\text{TQBF}(q) \iff \text{SEQUENTIAL-ANCHOR-SUFFICIENCY-CHECK}(\text{reduceTQBF}(q), \emptyset).$$

Hence TQBF many-one reduces to the sequential anchor query. ■

Theorem 5.11 (Sequential Minimum Query is PSPACE-complete). *The sequential minimum-sufficient-set query is PSPACE-complete.*

(L: DC46, DC48.)

Proof. Membership is in PSPACE. The query asks whether there exists a coordinate set I of size at most k such that I is sequentially sufficient. A nondeterministic polynomial-space procedure can guess I and then run the same polynomial-space verifier used for sequential sufficiency. Again, $\text{NPSpace} = \text{PSPACE}$.

Reduce TQBF to the $k = 0$ slice. There exists a sequentially sufficient set of size at most 0 iff the empty set itself is sequentially sufficient. The same construction yields PSPACE-hardness for SEQUENTIAL-MINIMUM-SUFFICIENT-SET. ■

Proposition 5.12 (Explicit Finite Search for Sequential Queries). *For finite instances, the development contains counted exhaustive-search procedures for sequential sufficiency, sequential anchor sufficiency, and sequential minimum sufficiency. Their certified step bounds are respectively $O(|S|^2)$, $O(|S|)$, and $O(2^n)$.*

(L: DC66-69, DC74-76.)

Proof. The sufficiency search scans for an agreeing pair of states with different optimal sets; the anchor search scans over candidate anchor states; and the minimum query scans the subset lattice. Each procedure admits a counted boolean search formulation with both a correctness theorem and an explicit step bound. ■

5.3 Tractable Subcases

Proposition 5.13 (Tractable Sequential Cases). *The sequential sufficiency problem is polynomial-time solvable under each of the following restrictions:*

1. full observability (MDP case),
2. bounded horizon,
3. tree-structured transition systems,
4. deterministic transitions.

(L: DQ12, DQ14, DQ45.)

Proof. Each restriction reduces policy search to dynamic programming over a polynomially bounded representation. ■

5.4 Bridge from Stochastic

The bridge from the stochastic regime is strict. A one-shot distribution can hide no temporal contingency; once dynamics are introduced, latent information can become decision-relevant later even when it appears irrelevant at time zero.

Proposition 5.14 (Stochastic Sufficiency Does Not Imply Sequential Sufficiency). *There exist instances where I is sufficient in the stochastic one-shot sense but fails to be sufficient once temporal dynamics are introduced.*

(L: DQ62.)

Proof. Construct a process in which the same one-step expected optimizer is preserved under I , but hidden-state memory affects future information states and therefore optimal policies. The temporal dependence blocks one-shot sufficiency transfer. ■

With the static, stochastic, and sequential classifications in place, the regime hierarchy is now complete. The next section consolidates that hierarchy into a single matrix, and the following section refines it with the encoding-sensitive ETH lower bounds that explain when exact certification remains feasible despite the worst-case class barriers.

6 Regime Matrix

The static, stochastic, and sequential formulations are the paper’s central organizing object. They ask the same underlying question about which coordinates matter for the decision, but they answer it in increasingly expressive models. In the stochastic regime, preservation and decisiveness become parallel exact predicates. Adding time then introduces temporal contingency. Read as a complexity matrix, adding probability introduces conditional-comparison hardness, and adding time lifts the core queries further to PSPACE.

6.1 Complexity Matrix

Regime / predicate	Base query	Minimum	Anchor
Static	coNP-complete	coNP-complete	Σ_2^P -complete
Stochastic preservation	P (explicit), bridge theorems	P (explicit); coNP-complete (full) / open (general)	P (explicit); Σ_2^P -complete (full) / open (general)
Stochastic decisiveness	P (explicit), PP-hard (succinct)	PP-hard, in NP^{PP}	PP-hard, in NP^{PP}
Sequential	PSPACE-complete	PSPACE-complete	PSPACE-complete

This table is the paper’s compact summary. The stochastic regime contains two parallel predicates with distinct complexity profiles: preservation and decisiveness. In the static regime, sufficiency and minimum sufficiency are coNP-complete and anchor sufficiency is Σ_2^P -complete. In the stochastic regime, preservation (stochastic sufficiency) has explicit-state tractability and bridge theorems back to static sufficiency and the optimizer quotient; the minimum and anchor preservation variants are also polynomial-time in explicit state, inherit coNP-completeness and Σ_2^P -completeness

under full support, and admit support-sensitive obstruction and bridge lemmas beyond full support, while the full general-distribution classification remains open. Decisiveness (stochastic decisiveness) has the sharper succinct-encoding classification: polynomial-time in the explicit-state model, PP-hardness in the succinct model, and paper-level NP^{PP} membership for the anchor and minimum variants. The artifact mechanizes the finite witness/checking schemas and reduction cores underlying those upper bounds. In the sequential regime, sufficiency, minimum, and anchor queries are PSPACE-complete.

Preservation Boundary. For stochastic sufficiency in the preservation sense, this paper proves explicit-state tractability for the preservation family (Theorem 4.5 and Theorem 4.13), static-to-stochastic bridge theorems (Propositions 4.6 and 4.7), quotient equivalence under full support (Proposition 4.8), full-support inheritance for the minimum and anchor variants (Theorem 4.11), and support-sensitive obstruction/bridge lemmas beyond full support (Proposition 4.14). Outside the full-support setting, this paper does *not* claim a succinct-encoding hardness classification for the preservation predicate or a full general-distribution classification for its minimum and anchor variants. The PP-hardness results in the stochastic row concern decisiveness and its variants, not preservation itself.

The decisiveness results combine standard hardness claims with mechanized upper-bound packages. The artifact verifies the finite witness/checking schemata and reduction cores used by the stochastic oracle-class arguments, while the corresponding oracle-class memberships are proved in the paper text by standard complexity-theoretic reasoning.

The stochastic anchor and minimum decisiveness queries are not presently collapsed to PP-completeness: Theorem 4.24 places them in NP^{PP} , while Theorems 4.21 and 4.22 give PP-hardness. The artifact verifies the finite witness/checking structure and explicit-state bounded-witness recoveries behind that upper-bound story, and it also packages existential-majority hardness/completeness surrogates for the anchor and decisiveness query families (L: EH2-10.) The oracle-class memberships themselves remain paper-level proofs under the succinct encoding. By contrast, the sequential anchor and minimum variants admit paper-level PSPACE membership via guess-and-verify over anchors or coordinate subsets together with the sequential sufficiency verifier.

Theorem 6.1 (Regime-Sensitive Complexity Matrix). *Under the encoding conventions of Section 2.4, the classification is as follows. In the static regime, sufficiency and minimum sufficiency are coNP-complete and anchor sufficiency is Σ_2^P -complete. In the stochastic regime, preservation (stochastic sufficiency) has a polynomial-time base query under explicit-state encoding with proved bridges back to static sufficiency and the optimizer quotient; the minimum and anchor preservation variants are also polynomial-time under explicit-state encoding, are coNP-complete and Σ_2^P -complete under full support, and satisfy the support-sensitive partial results of Proposition 4.14. Decisiveness (stochastic decisiveness) has a polynomial-time base query under explicit-state encoding and is PP-hard under the succinct encoding; the anchor and minimum decisiveness queries are PP-hard and lie in NP^{PP} , with those oracle-class memberships proved in the paper text and their finite witness/checking cores mechanized. In the sequential regime, sufficiency, minimum, and anchor queries are PSPACE-complete.*

(L: DC100, DQ1, DQ17-19, OV1-2, OV11-12, EH4-10.)

Proof. The matrix entries are exactly Theorems 3.4, 3.5, 3.6, 4.5, 4.11, 4.13, Proposition 4.14, 4.19, 4.21, 4.22, 4.24, 5.8, 5.10, and 5.11, together with the stochastic sufficiency bridge package. The strict regime comparisons are isolated separately in Theorems 6.2 and 6.3. ■

6.2 Strict Regime Gaps

Theorem 6.2 (Static-to-Stochastic Complexity Gap). *Assuming $\text{coNP} \neq \text{PP}$, the succinct stochastic decisiveness classification is strictly harder in worst-case complexity than the static sufficiency classification.*

(L: DQ18.)

Proof. Static sufficiency checking is coNP -complete (Theorem 3.4), while stochastic decisiveness is PP -hard under the succinct encoding (Theorem 4.19). Under the assumption $\text{coNP} \neq \text{PP}$, this yields a strict complexity gap between the two classifications. ■

Theorem 6.3 (Stochastic-to-Sequential Complexity Gap). *Assuming $\text{PP} \neq \text{PSPACE}$, the sequential sufficiency classification is strictly harder in worst-case complexity than the stochastic decisiveness classification.*

(L: DQ19.)

Proof. Stochastic decisiveness is PP -hard under the succinct encoding (Theorem 4.19), while sequential sufficiency is PSPACE -complete (Theorem 5.8). The class gap yields a strict complexity separation between the two classifications under the assumption $\text{PP} \neq \text{PSPACE}$. ■

6.3 Integrity-Resource Bounds

The regime matrix has a methodological consequence as well: as one moves upward in the hierarchy, exact certification becomes less compatible with fixed polynomial resource budgets.

Proposition 6.4 (Abstention Frontier Expands with Regime Complexity). *For polynomial-time exact certifiers that abstain whenever they cannot certify correctness within budget, the set of instances requiring abstention expands along*

$$\text{static} \rightarrow \text{stochastic} \rightarrow \text{sequential}.$$

(L: IC32, DQ2.)

Proof. As the decision predicate moves from coNP to PP to PSPACE hardness, exact worst-case certification requires strictly stronger resources. Fixed polynomial resources therefore force abstention on a larger instance family in higher regimes. ■

Forward-looking perspective. The regime hierarchy extends to richer models: partially observable settings, multi-agent decision problems, and game-theoretic regimes would introduce additional layers of complexity such as knowledge reasoning or equilibrium computation. The optimizer-quotient framework extends to those regimes, though their detailed classification remains open.

7 Encoding Dichotomy and ETH Lower Bounds

The regime hierarchy identifies where exact relevance certification sits in the standard complexity landscape. This section extracts the complementary algorithmic message: under the encodings of Section 2.4, there is a strong contrast between explicit-state instances whose true relevant support is small and succinct instances whose relevant support is extensive. The former admit direct algorithms; the latter inherit exponential lower bounds under ETH. The practical takeaway is

cross-model rather than single-model: logarithmic relevant support can still be certified exactly in polynomial time under explicit encodings, while linear relevant support under succinct encodings already forces ETH-conditioned exponential cost.

Model note. Part 1 is an explicit-state upper bound (time polynomial in $|S|$). Part 2 is a succinct-encoding lower bound under ETH (time exponential in n). The encodings are defined in [Section 2.4](#).

Encoding	Support regime	Complexity consequence	Source
Explicit-state	$k^* = O(\log N)$	exact certification is polynomial in N by projection enumeration	Theorem 7.1(1)
Succinct	$k^* = \Omega(n)$	exact certification inherits an ETH-conditioned $2^{\Omega(n)}$ lower bound	Theorem 7.1(2)

Boundary summary. The tractable-hard contrast is controlled jointly by encoding and by the size of the true sufficient support.

Theorem 7.1 (Encoding-Sensitive Contrast). *Let $\mathcal{D} = (A, X_1, \dots, X_n, U)$ be a decision problem with $|S| = N$ states. Let k^* be the size of the minimal sufficient set.*

1. **Logarithmic case (explicit-state upper bound):** *If $k^* = O(\log N)$, then SUFFICIENCY-CHECK is solvable in polynomial time in N under the explicit-state encoding.*
2. **Linear case (succinct lower bound under ETH):** *If $k^* = \Omega(n)$, then SUFFICIENCY-CHECK requires time $\Omega(2^{n/c})$ for some constant $c > 0$ under the succinct encoding (assuming ETH).*

(L: [DQ26](#), [DQ29-30](#).)

Proof. Part 1 (Logarithmic case): If $k^* = O(\log N)$, then the number of distinct projections $|S_{I^*}|$ is at most $2^{k^*} = O(N^c)$ for some constant c . Under the explicit-state encoding, we enumerate all projections and verify sufficiency in polynomial time.

Part 2 (Linear case): We establish this by a standard ETH transfer under the succinct encoding. ■

7.1 The ETH Reduction Chain

The lower bound in Part 2 of Theorem 7.1 follows from a short standard ETH transfer. Under ETH, 3-SAT requires $2^{\Omega(n)}$ time. Negation gives a linear-time reduction from 3-SAT to TAUTOLOGY. The static reduction of Theorem 3.4 then maps an n -variable tautology instance to a sufficiency instance with the same asymptotic coordinate count. Therefore, if SUFFICIENCY-CHECK were solvable in $2^{o(n)}$ time under the succinct encoding, then so would TAUTOLOGY and hence 3-SAT, contradicting ETH.

Definition 7.2 (Exponential Time Hypothesis (ETH)). There exists a constant $\delta > 0$ such that 3-SAT on n variables cannot be solved in time $O(2^{\delta n})$ [16].

Proposition 7.3 (Tight Constant). *The reduction in Theorem 3.4 preserves the number of variables up to an additive constant: an n -variable formula yields an $(n + 1)$ -coordinate decision problem. Therefore, the constant c in the $2^{n/c}$ lower bound is asymptotically 1:*

$$\text{SUFFICIENCY-CHECK requires time } \Omega(2^{\delta(n-1)}) = 2^{\Omega(n)} \text{ under ETH}$$

where δ is the ETH constant for 3-SAT.

(L: DQ27, DQ30.)

Proof. The TAUTOLOGY reduction (Theorem 3.4) constructs:

- State space $S = \{\text{ref}\} \cup \{0, 1\}^n$ with $n + 1$ coordinates (one extra for the reference state)
- Query set $I = \emptyset$

When φ has n variables, the constructed problem has $n + 1$ coordinates. The asymptotic lower bound is $2^{\Omega(n)}$ with the same constant δ from ETH. ■

7.2 Cross-Model Contrast

Corollary 7.4 (Cross-Model Contrast for Exact Certification). *Across the explicit-state and succinct encodings of Section 2.4, exact certification exhibits the following contrast:*

- If $k^* = O(\log N)$, *SUFFICIENCY-CHECK* is polynomial in N under the explicit-state encoding
- If $k^* = \Omega(n)$, *SUFFICIENCY-CHECK* is exponential in n under ETH in the succinct encoding

For Boolean coordinate spaces ($N = 2^n$), one has $\log N = n$. So the explicit-state upper bound should be read as polynomial in the explicit table size when $k^* = O(n)$, while the succinct lower bound is exponential in the succinct variable count when $k^* = \Omega(n)$. The comparison is therefore between two encodings of the same Boolean family, not between $O(\log n)$ and $\Omega(n)$ inside a single model.

(L: DQ3, DQ26.)

Proof. The logarithmic case (Part 1 of Theorem 7.1) gives polynomial time when $k^* = O(\log N)$. More precisely, when $k^* \leq c \log N$ for constant c , the algorithm runs in time $O(N^c \cdot \text{poly}(n))$.

The linear case (Part 2) gives exponential time when $k^* = \Omega(n)$.

The transition point inside the explicit-state bound is where $2^{k^*} = N^{k^*/\log N}$ stops being polynomial in N , i.e., when $k^* = \omega(\log N)$. For Boolean coordinate spaces, however, the present theorem should be read only as a cross-encoding contrast, because $\log N = n$ and the succinct lower bound is parameterized directly by the succinct variable count. ■

Remark 7.5 (Asymptotic Tightness Within Each Encoding). Under ETH, the lower bound is asymptotically tight in the succinct encoding. The explicit-state upper bound is tight in the sense that it matches enumeration complexity in N .

This encoding-sensitive contrast explains why exact simplification can be tractable on some instance families and computationally prohibitive on others. When the true sufficient support is tiny and the state space is explicit, exhaustive certification can still be polynomial. When the true support is extensive and the problem is given succinctly, the same certification task inherits a worst-case $2^{\Omega(n)}$ lower bound under ETH in addition to the class-level coNP hardness statement.

Interpretation. The ETH chain composes with the structural rank viewpoint introduced earlier. When relevance is extensive, the same objects that drive the regime hierarchy also force the exponential obstruction in the succinct model. In that sense, the dichotomy theorem complements the hierarchy rather than standing apart from it: the hierarchy locates exact certification in the standard class landscape, while the dichotomy explains when explicit structure can still make exact certification feasible.

Remark 7.6 (Circuit Model Formalization). The ETH lower bound is stated in the succinct circuit encoding of Section 2.4, where the utility function $U : A \times S \rightarrow \mathbb{R}$ is represented by a Boolean circuit computing $\mathbf{1}[U(a, s) > \theta]$ for threshold comparisons. In this model:

- The input size is the circuit size m , not the state space size $|S| = 2^n$
- A 3-SAT formula with n variables and c clauses yields a circuit of size $O(n + c)$
- The reduction preserves instance size up to constant factors: $m_{\text{out}} \leq 3 \cdot m_{\text{in}}$

This linear size preservation is essential for ETH transfer. In the explicit enumeration model (where S is given as a list), the reduction would blow up the instance size exponentially, precluding ETH-based lower bounds. The circuit model is standard in fine-grained complexity and matches practical representations of decision problems.

8 Tractable Special Cases

We distinguish the encodings of Section 2.4. The tractability results below state the model assumption explicitly.

Our theory provides comprehensive coverage of tractable subcases. The artifact certifies explicit decision procedures for several families, and also certifies paper-specific reductions and complexity transfers connecting structurally interesting restrictions to standard polynomial-time algorithmic backbones. Each subcase removes one of the structural sources of hardness in exact certification: unrestricted action comparison, cross-coordinate interaction, high-width dependency structure, or unnecessary state-space multiplicity.

Restriction	Why it helps	Complexity
Bounded actions	brute-force enumeration is polynomial in $ S $ when $ A $ constant	$O(S ^2 \cdot k^2)$
Separable utility	optimal action independent of state	$O(1)$
Low tensor rank	factored evaluation avoids full state enumeration	$O(A \cdot R \cdot n)$
Tree structure	dynamic programming over dependency tree suffices	$O(n \cdot A \cdot k_{\max})$
Bounded treewidth	interaction checking reduces to low-width CSP	$O(n \cdot k^{w+1})$
Coordinate symmetry	orbit types replace raw state pairs	$O\left(\binom{d+k-1}{k-1}^2\right)$

Tractability map. Each restriction above removes a structural hardness source. Formal definitions for all cases are retained in the subsections below.

Theorem 8.1 (Tractable Subcases). *The following structurally interesting tractable subcases are all mechanized in artifact. The artifact certifies paper-specific reductions and complexity transfers*

connecting each to standard polynomial-time algorithmic backbones. Formal definitions and trivial cases (single action, bounded state space, separable utility, dominance) are retained in the subsections below.

1. **Low tensor rank:** The utility admits a rank- R decomposition $U(a, s) = \sum_{r=1}^R w_r \cdot f_r(a) \cdot \prod_i g_{ri}(s_i)$. The artifact certifies reduction to factored computation with bound $O(|A| \cdot R \cdot n)$. (L: [DQ77](#).)
2. **Tree-structured dependencies:** Coordinates are ordered such that s_i depends only on $(s_1, \dots, s_{i-1})$. The artifact certifies an explicit sufficiency checker for this class. (L: [DQ63](#).)
3. **Bounded treewidth:** The interaction graph has treewidth $\leq w$ and utility factors over edges. The artifact certifies reduction to bounded-treewidth CSP together with bound $O(n \cdot k^{w+1})$ inherited from standard CSP algorithm. (L: [DQ81](#).)
4. **Coordinate symmetry:** Utility is invariant under coordinate permutations. The artifact certifies reduction to orbit-type representatives together with resulting orbit-count bound $O\left(\binom{d+k-1}{k-1}\right)^2$. (L: [DQ85](#).)

The following subsections provide the formal definitions and reduction theorems for each tractable subcase.

8.1 Bounded Actions

When the action set is bounded, brute-force enumeration becomes polynomial.

Definition 8.2 (Bounded Action Problem). A decision problem has *bounded actions* with parameter k if $|A| \leq k$ for some constant k independent of the state space size.

Theorem 8.3 (Bounded Actions Tractability). If $|A| \leq k$ for constant k , then *SUFFICIENCY-CHECK* runs in $O(|S|^2 \cdot k^2)$ time.

(L: [DQ51](#), [DQ72](#).)

Proof. Given the full table of $U(a, s)$:

1. Compute $\text{Opt}(s)$ for all $s \in S$ in $O(|S| \cdot k)$ time.
2. For each pair (s, s') with $s_I = s'_I$, compare $\text{Opt}(s)$ and $\text{Opt}(s')$ in $O(k^2)$ time.
3. Total: $O(|S|^2)$ pairs $\times O(k^2)$ comparisons $= O(|S|^2 \cdot k^2)$.

When k is constant, this is polynomial in $|S|$. ■

8.2 Separable Utility (Rank 1)

Separable utility decouples actions from states, making sufficiency trivial.

Definition 8.4 (Separable Utility). A utility function is *separable* if there exist functions $f : A \rightarrow \mathbb{R}$ and $g : S \rightarrow \mathbb{R}$ such that $U(a, s) = f(a) + g(s)$ for all $(a, s) \in A \times S$.

Theorem 8.5 (Separable Tractability). If utility is separable, then $I = \emptyset$ is always sufficient.

(L: [DQ57](#), [DQ74](#).)

Proof. If $U(a, s) = f(a) + g(s)$, then:

$$\text{Opt}(s) = \arg \max_{a \in A} [f(a) + g(s)] = \arg \max_{a \in A} f(a)$$

The optimal action depends only on f , not on s . Hence $\text{Opt}(s) = \text{Opt}(s')$ for all s, s' , making any I (including $\emptyset$) sufficient. ■

8.3 Low Tensor Rank

Utility functions with low tensor rank admit efficient factored computation, generalizing separable utility.

Definition 8.6 (Tensor Rank Decomposition). A utility function $U : A \times ((i : \text{Fin } n) \rightarrow X_i) \rightarrow \mathbb{R}$ has *tensor rank* R if there exist weights $w_1, \dots, w_R \in \mathbb{R}$, action factors $f_r : A \rightarrow \mathbb{R}$, and coordinate factors $g_{ri} : X_i \rightarrow \mathbb{R}$ such that:

$$U(a, s) = \sum_{r=1}^R w_r \cdot f_r(a) \cdot \prod_{i=1}^n g_{ri}(s_i)$$

Remark 8.7. Separable utility (Definition 8.4) is the special case $R = 1$ with $w_1 = 1$, $f_1(a) = f(a)$, and $\prod_i g_{1i}(s_i) = g(s)$.

Theorem 8.8 (Low Tensor Rank Reduction). *If utility has tensor rank R , the artifact certifies a reduction of the relevant optimization step to factored computation with bound $O(|A| \cdot R \cdot n)$.*

(L: [DQ75-77](#).)

Proof. The artifact certifies three ingredients. First, bounded-rank tensor contraction admits a polynomial bound (L: [DQ75](#).) Second, the low-rank utility representation reduces the relevant optimization step to factored computation in $O(|A| \cdot R \cdot n)$ steps (L: [DQ76-77](#).) Third, these are exactly the paper-specific ingredients needed to invoke the standard tensor-network algorithms cited in the text. Thus the formal support here is a certified reduction-and-transfer statement rather than an end-to-end formalization of the tensor-network backbone itself. ■

8.4 Tree-Structured Dependencies

When coordinate dependencies form a tree, dynamic programming yields polynomial-time sufficiency checking.

Definition 8.9 (Tree-Structured Dependencies). A decision problem has *tree-structured dependencies* if there exists an ordering of coordinates $(1, \dots, n)$ such that the utility decomposes as:

$$U(a, s) = \sum_{i=1}^n u_i(a, s_i, s_{\text{parent}(i)})$$

where $\text{parent}(i) < i$ for all $i > 1$, and the root term depends only on (a, s_1) .

Theorem 8.10 (Tree Structure Tractability). *If dependencies are tree-structured with explicit local tables, SUFFICIENCY-CHECK runs in $O(n \cdot |A| \cdot k_{\max})$ time where $k_{\max} = \max_i |X_i|$.*

(L: [DQ63](#), [DQ78](#).)

Proof. Dynamic programming on the tree order:

1. Process coordinates in order $i = n, n - 1, \dots, 1$.
2. For each node i and each value of its parent coordinate, compute the set of actions optimal for some subtree assignment.
3. Combine child summaries with local tables in $O(|A| \cdot k_{\max})$ per node.
4. Total: $O(n \cdot |A| \cdot k_{\max})$.

A coordinate is relevant iff varying its value changes the optimal action set. ■

8.5 Bounded Treewidth

The tree-structured case generalizes to bounded treewidth interaction graphs via CSP reduction. This directly parallels backdoor tractability methods for CSPs [2], where finding a small backdoor to a tractable class enables efficient solving.

Definition 8.11 (Interaction Graph). Given a pairwise utility decomposition, the *interaction graph* $G = (V, E)$ has:

- Vertices $V = \{1, \dots, n\}$ (one per coordinate)
- Edges $E = \{(i, j) : i \text{ and } j \text{ interact in utility}\}$

Definition 8.12 (Pairwise Utility). A utility function is *pairwise* if it decomposes as:

$$U(a, s) = \sum_{i=1}^n u_i(a, s_i) + \sum_{(i,j) \in E} u_{ij}(a, s_i, s_j)$$

with unary terms u_i and binary terms u_{ij} given explicitly.

Theorem 8.13 (Bounded Treewidth Reduction). *If the interaction graph has treewidth $\leq w$ and utility is pairwise with explicit factors, the artifact certifies a reduction to bounded-treewidth CSP together with standard complexity bound $O(n \cdot k^{w+1})$, where $k = \max_i |X_i|$. The reduction bridges to established CSP algorithms [5, 11, 2].*

(L: DQ79-81.)

Proof. The artifact certifies the paper-specific reduction from pairwise sufficiency checking to a CSP on the interaction graph (L: DQ80.) It also certifies the transfer of the standard bounded-treewidth complexity bound (L: DQ79, DQ81.) Thus formal support here is a certified reduction-and-transfer theorem: the bounded-treewidth algorithm itself is standard (drawing on backdoor tractability methods [2]), while the paper-specific bridge to that algorithm is mechanized. ■

8.6 Coordinate Symmetry

When utility is invariant under coordinate permutations, the effective state space collapses to orbit types.

Definition 8.14 (Dimensional State Space). A *dimensional state space* with alphabet size k and dimension d is $S = \{0, \dots, k - 1\}^d$, the set of d -tuples over a k -element alphabet.

Definition 8.15 (Symmetric Utility). A utility function on a dimensional state space is *symmetric* if it is invariant under coordinate permutations: for all permutations $\sigma \in \mathfrak{S}_d$ and states $s \in S$,

$$U(a, s) = U(a, \sigma \cdot s) \quad \text{where } (\sigma \cdot s)_i = s_{\sigma^{-1}(i)}$$

Definition 8.16 (Orbit Type). The *orbit type* of a state $s \in \{0, \dots, k-1\}^d$ is the multiset of its coordinate values:

$$\text{orbitType}(s) = \{\{s_1, s_2, \dots, s_d\}\}$$

Two states have the same orbit type iff they lie in the same orbit under coordinate permutation.

Theorem 8.17 (Orbit Type Characterization). *For dimensional state spaces, two states s, s' have equal orbit types if and only if there exists a coordinate permutation σ such that $\sigma \cdot s = s'$.*

(L: DQ82.)

Proof. If: Permuting coordinates preserves the multiset of values. **Only if:** If s and s' have the same multiset of values, we can construct a permutation mapping each occurrence in s to the corresponding occurrence in s' . ■

Theorem 8.18 (Symmetric Sufficiency Reduction). *Under symmetric utility, optimal actions depend only on orbit type:*

$$\text{orbitType}(s) = \text{orbitType}(s') \implies \text{Opt}(s) = \text{Opt}(s')$$

Thus *SUFFICIENCY-CHECK* reduces to checking pairs with different orbit types.

(L: DQ83-84.)

Proof. If $\text{orbitType}(s) = \text{orbitType}(s')$, then by Theorem 8.17 there exists σ with $\sigma \cdot s = s'$. By symmetry, $U(a, s) = U(a, s')$ for all a , so $\text{Opt}(s) = \text{Opt}(s')$.

For sufficiency, we need only check pairs (s, s') agreeing on I with *different* orbit types; same-orbit pairs are automatically equal. ■

Theorem 8.19 (Symmetric Complexity Bound). *Under symmetric utility, the artifact certifies that the number of orbit types is bounded by the stars-and-bars count:*

$$|\text{OrbitTypes}| = \binom{d+k-1}{k-1}$$

and that sufficiency checking reduces to cross-orbit comparisons. Thus the effective number of representative pair checks is at most $\binom{d+k-1}{k-1}^2$, which is polynomial in d for fixed k .

(L: DQ56, DQ85.)

Proof. An orbit type is a multiset of d values from $\{0, \dots, k-1\}$, equivalently a k -tuple of non-negative integers summing to d . By stars-and-bars, the count is $\binom{d+k-1}{k-1}$. The mechanized content here is the orbit-type reduction and the resulting representative-count bound; the resulting polynomial-time conclusion then follows from this certified compression of the comparison space.

For fixed k , this is $O(d^{k-1})$, polynomial in d . ■

8.7 Practical Implications

The formally developed tractable subcases correspond to common modeling scenarios. The structurally nontrivial cases each remove a distinct source of hardness:

- **Bounded actions:** Small action sets (e.g., binary decisions, few alternatives). Brute-force enumeration is polynomial when $|A|$ is constant.
- **Separable utility:** Additive cost models, linear utility functions. The optimizer is independent of state, making any coordinate set sufficient.
- **Low tensor rank:** Factored preference models, low-rank approximations in recommendation systems or multi-attribute utility theory.
- **Tree structure:** Hierarchical decision processes, causal models with chain or tree dependency. Dynamic programming replaces exhaustive search.
- **Bounded treewidth:** Spatial models, graphical models with limited interaction width. CSP algorithms on low-treewidth graphs handle the sufficiency check.
- **Coordinate symmetry:** Exchangeable features, order-invariant utilities (e.g., set-valued inputs). Orbit-type representatives collapse the comparison space.

Simple edge cases also yield immediate tractability without structural argument: single action ($|\text{Opt}| = |A|$ trivially), bounded state space (exhaustive search is polynomial in $|S|$), strict dominance (one action always weakly optimal), and constant optimal set ($\text{Opt}(s)$ independent of s).

For problems given in the succinct encoding without these structural restrictions, the hardness results of Section 3 apply, justifying heuristic approaches.

9 Implications for Exact Certification

The regime hierarchy matters for more than class labels. The same exact question, which coordinates can be omitted without changing the decision boundary, reappears when one tries to simplify models, build reliable certifiers, check witnesses, approximate exact minimizers, or compress a central interface. This section collects those consequences in one place. The common lesson is that exact relevance is expensive to certify and easy to displace rather than remove. One downstream consequence is a trilemma for exact certifiers in the hard regime: once exact certification is hard, every interface-compression story must be read under the possibility of abstention, weakened guarantees, or overclaiming.

9.1 Exact Simplification and Over-Specification

Proposition 9.1 (Configuration Simplification as SUFFICIENCY-CHECK). *Let $P = \{p_1, \dots, p_n\}$ be configuration parameters and let B be a finite set of observable behaviors. Suppose each configuration state determines the subset of behaviors that remain feasible or optimal. Then the question*

“Does parameter subset $I \subseteq P$ preserve all decision-relevant behavior?”

is an instance of SUFFICIENCY-CHECK.

(L: CR1, CT12.)

Proof. Construct a decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ by taking actions $A = B$, coordinate domains X_i equal to the domains of the parameters p_i , and defining $U(b, s) = 1$ exactly when behavior b is realized or admissible under configuration state s . Then $\text{Opt}(s)$ is the behavior set induced by s , and coordinate subset I is sufficient exactly when agreement on the parameters in I forces agreement on the induced optimal-behavior set. ■

Concrete case study. Consider a simplified service configuration with three parameters: cache mode $p_1 \in \{\text{off}, \text{on}\}$, retry policy $p_2 \in \{1, 3\}$, and replica count $p_3 \in \{1, 2\}$. Let the observable behavior set be $B = \{\text{latency-ok}, \text{write-safe}, \text{cost-ok}\}$. Suppose enabling the cache affects only latency, increasing retries affects only write safety, and replica count affects write safety and cost. Then Theorem 9.1 turns the simplification question into an exact sufficiency question on the induced decision problem: which parameter subsets preserve the optimal-behavior set? In this toy instance, if the target simplification must preserve write safety and cost but is indifferent to latency, the exact behavior-preserving core is $\{p_2, p_3\}$, while p_1 is irrelevant to that target behavior set. The example is small enough to inspect by hand, but it is already an instance of the general Lean-backed reduction from behavior preservation to sufficiency. (L: CR1, CT12.)

9.1.1 Formalized toy POMDP example

Definition 9.2 (One-step POMDP). A one-step POMDP is a tuple (S, A, O, p, o, r) where S, A, O are finite sets of states, actions, and observations, $p \in \Delta(S)$ is a prior distribution, $o : S \rightarrow O$ is the observation map, and $r : A \times S \rightarrow \mathbb{R}$ is the immediate reward function. The horizon is one, so the utility of taking action a in true state s is $r(a, s)$.

Write $\text{Opt}_{\text{full}}(s) := \arg \max_{a \in A} r(a, s)$ for the full-information optimal-action set and, for an observation value $o \in O$, set

$$\text{Opt}_{\text{obs}}(o) := \arg \max_{a \in A} \mathbb{E}_{S \sim p}[r(a, S) \mid o(S) = o].$$

Let $g : O \rightarrow O'$ be any coarsening of observations and write $\phi := g \circ o : S \rightarrow O'$ for the induced coarse signal.

Proposition 9.3 (Reduction to stochastic preservation). *Let (S, A, O, p, o, r) be a one-step POMDP and let $\phi : S \rightarrow O'$ be as above. Define the decision problem $\mathcal{D} = (A, S, U)$ with $U(a, s) := r(a, s)$ and state distribution p . Then ϕ is stochastically preserving for $\mathcal{D}$ in the sense of Section 4 if and only if for every $s \in S$ we have*

$$\text{Opt}_{\text{full}}(s) = \text{Opt}_{\text{obs}}(\phi(s)).$$

In particular, checking whether the coarse observation ϕ preserves Bayes-optimal actions is exactly an instance of our stochastic preservation predicate for $\mathcal{D}$.

Proof. By construction $U(a, s) = r(a, s)$ so the full-information optimal-action set in $\mathcal{D}$ equals $\text{Opt}_{\text{full}}(s)$. The coarse-induced optimizer for ϕ is, by definition, the set of actions maximizing the conditional expectation of U given the coarse signal. That conditional expectation is exactly the Bayes expectation used to define Opt_{obs} . Therefore the preservation condition from Section 4 (equality of full-information and coarse-induced optimizers on every state) is equivalent to the displayed identity. ■

(L: EX1.)

Worked numeric instance. Consider the one-step POMDP with states $S = \{s_1, s_2\}$, actions $A = \{a, b\}$, prior $p(s_1) = p(s_2) = 1/2$, observation map $o(s_1) = o_1, o(s_2) = o_2$, and reward table

$$r(a, s_1) = 2, \quad r(b, s_1) = 1, \quad r(a, s_2) = 0, \quad r(b, s_2) = 3.$$

Let $g : O \rightarrow O'$ be the constant map that coarsens both observations to a single symbol. Then for the coarse signal $\phi = g \circ o$ the conditional expected returns are

$$\mathbb{E}[r(a, S) \mid \phi] = \frac{1}{2}(2 + 0) = 1, \quad \mathbb{E}[r(b, S) \mid \phi] = \frac{1}{2}(1 + 3) = 2.$$

Thus the coarse-induced optimizer is $\{b\}$ while the full-information optimizer at s_1 is $\{a\}$, so ϕ does not preserve Bayes-optimal actions for this instance. This tiny numeric example illustrates how coarse observations can change the optimal-action correspondence and therefore fail the stochastic preservation predicate. (L: EX3.)

This gives a direct, elementary encoding of one-step planning questions as stochastic preservation instances. The explicit-state encoding of the POMDP yields an explicit-state preservation instance, while a succinct encoding of transition or observation structure yields a succinct encoding of the corresponding decision problem, explaining how the same modelling choices generate the different complexity behaviors reported in Section 4.

9.1.2 Formalized reinforcement-learning hyperparameter example

Fix a finite environment set E and finite hyperparameter domains $X_\alpha, X_\gamma, X_\epsilon$. Let

$$H := X_\alpha \times X_\gamma \times X_\epsilon$$

be the set of hyperparameter configurations. For each environment $e \in E$ write $f_e : H \rightarrow \mathbb{R}$ for the expected return of configuration $h \in H$.

Define the derived decision problem

$$\mathcal{D}_{\text{hp}} = (A, H, U), \quad A := E, \quad U(e, h) := f_e(h).$$

Here states are hyperparameter configurations and actions are environments; the utility records return when configuration h is evaluated under environment e .

Let $\pi : H \rightarrow H' := X_\alpha \times X_\epsilon$ be the projection that drops the γ coordinate. Call γ redundant for the tuning task if for every environment e the set of maximizers

$$\text{Opt}_e := \arg \max_{h \in H} f_e(h)$$

projects under π to the set of maximizers of the collapsed objective

$$g_e(h') := \max_{\gamma \in X_\gamma} f_e(h', \gamma), \quad h' \in H'.$$

Proposition 9.4 (Reduction to a static preservation check). *With notation as above, the statement " γ is redundant for tuning" is equivalent to a static preservation property for the decision problem $\mathcal{D}_{\text{hp}}$ under the action-space projection π . Concretely, γ is redundant if and only if for every $e \in E$ and every $h \in H$ we have*

$$h \in \text{Opt}_e \iff \exists h_0 \in \text{Opt}_e \text{ with } \pi(h_0) = \pi(h),$$

which is exactly the condition that the projection π preserves the optimal-action correspondence when the decision problem is read with environments as the existential choices.

Proof. By definition γ is redundant when every full maximizer projects to a maximizer of the collapsed objective and conversely every maximizer of the collapsed objective has a witness in the full space; the displayed equivalence is a restatement of that property. The left-to-right implication is immediate from the definition of g_e , and the right-to-left implication follows because if some projected maximizer has a preimage that attains the collapsed optimum, then there exists a full configuration achieving the same maximum in the original objective. Rewriting these statements as equalities of optimal-action sets for the derived decision problem $\mathcal{D}_{\text{hp}}$ yields the claimed reduction to a static-style preservation check. ■

(L: [E12](#).)

The construction above is explicit and elementary. It shows that hyperparameter redundancy questions can be phrased exactly as decision-preservation predicates within our static regime, and that succinct encodings of the environment family or return functions translate into succinct encodings of the corresponding static instance. In practice this lets one import the static-regime hardness and tractability statements directly into the hyperparameter setting.

(L: [CR1](#), [CT12](#).)

Corollary 9.5 (No General-Purpose Exact Configuration Minimizer). *Assuming $P \neq \text{coNP}$, there is no polynomial-time general-purpose procedure that takes an arbitrary succinctly encoded configuration problem and always returns a smallest behavior-preserving parameter subset.*

(L: [DQ36-37](#).)

Proof. Such a procedure would solve MINIMUM-SUFFICIENT-SET in polynomial time for arbitrary succinctly encoded instances, contradicting Theorem [3.5](#) under the assumption $P \neq \text{coNP}$. ■

Proposition 9.6 (Model-Conditional Rationality of Over-Specification). *Consider a design process in which:*

1. *removing a parameter requires certifying that it is irrelevant,*
2. *exact irrelevance certification is charged according to the computational model of Section [2.4](#), and*
3. *carrying an extra parameter incurs only linear or otherwise low-order maintenance overhead.*

Then, for sufficiently expressive succinctly encoded instances, retaining extra parameters can minimize total cost under the stated cost model.

(L: [DQ26](#), [HD14](#).)

Proof. By Theorems [3.4](#) and [3.5](#), exact sufficiency certification and exact minimization are coNP-complete in the general succinct regime. By Section [7](#), this hardness is accompanied by exponential lower bounds under ETH in the linear-support regime. If the cost of carrying extra parameters grows only linearly while exact minimization inherits worst-case exponential cost, then beyond a problem-dependent threshold the latter dominates the former. Under that cost model, retaining extra parameters minimizes total cost. ■

Remark 9.7. This does not rule out useful simplification tools. It rules out a fully general exact minimizer with worst-case polynomial guarantees. The structured regimes isolated in Section [8](#) remain viable precisely because they restrict the source of hardness.

9.2 Regime-Limited Exact Certification

Proposition 9.8 (Exact Certification Competence Depends on Regime). *Within the model of this paper, exact certification competence is regime-dependent. In the general succinct regime, exact relevance certification and exact minimization inherit the hardness results of Sections 3 and 7. In the structured regimes of Section 8, exact certification becomes available in polynomial time.*

(L: DQ36, DQ40, DQ44.)

Proof. The negative side is given by Theorems 3.4 and 3.5, together with the ETH-conditioned lower bounds summarized in Section 7. The positive side is given by the tractability results of Section 8, which provide explicit polynomial-time certification procedures under structural restrictions. ■

The next three subsections sharpen that basic regime distinction in different directions: exact reliability under polynomial budgets, witness-checking and approximation limits, and the cost of compressing a central interface while leaving exact relevance unresolved.

9.3 Integrity, Competence, and the Trilemma of Exact Certification

Once exact certification is hard, the direct systems question is not only whether the predicate can be computed, but whether a solver can stay exact, non-abstaining, and polynomial-budgeted on the whole hard regime. The result below is a direct complexity-theoretic corollary: those three goals are incompatible. We refer to this incompatibility as the *trilemma of exact certification*.

Definition 9.9 (Certifying Solver). For exact sufficiency on a declared regime Γ , a certifying solver is a pair (Q, V) where:

- Q maps each instance to either ABSTAIN or a candidate verdict with witness,
- V verifies witnesses in polynomial time (in the declared encoding length).

Definition 9.10 (Integrity). A certifying solver is *integrity-preserving* if every non-abstaining output is accepted by V , and every accepted output is correct for the declared exact relation.

Definition 9.11 (Competence). A certifying solver is *competent* on Γ if it is integrity-preserving, non-abstaining on all in-scope instances, and polynomial-budget compliant on all in-scope instances.

Definition 9.12 (Exact Reliability Claim). An *exact reliability claim* on Γ is the conjunction of universal non-abstaining coverage, correctness, and polynomial-budget compliance for exact sufficiency.

Proposition 9.13 (Integrity and Competence Separation). *Integrity and competence are distinct: integrity constrains what can be asserted, while competence adds full coverage under resource bounds.*

(L: DQ10, IC33.)

Proof. The always-abstain solver is integrity-preserving (it makes no uncertified assertion) but fails competence whenever the in-scope set is non-empty. ■

Theorem 9.14 (Integrity-Resource Bound (Conditional)). *Fix a regime Γ whose exact-sufficiency decision problem is coNP-complete under the declared encoding. Under $P \neq \text{coNP}$, no solver is simultaneously integrity-preserving and competent on Γ for exact sufficiency.*

(L: IC34, DQ31.)

Proof. Assume such a solver (Q, V) exists. We build a polynomial-time decider for exact sufficiency on Γ .

Given an input instance $x \in \Gamma$, run $Q(x)$. Competence implies that Q never abstains on in-scope instances and always halts within polynomial budget. Let y be the returned verdict with witness. Now run the polynomial-time verifier $V(x, y)$.

Integrity implies two things: every non-abstaining output produced by Q is accepted by V , and every accepted output is correct for the declared exact-sufficiency relation. Therefore the combined procedure returns the correct exact verdict on every instance in Γ and runs in polynomial time.

This yields a polynomial-time algorithm for a coNP-complete problem on Γ , hence $\text{coNP} \subseteq \text{P}$. Under $\text{P} \neq \text{coNP}$, this is impossible. Therefore no solver is simultaneously integrity-preserving and competent on Γ . ■

Corollary 9.15 (Exact Reliability Impossibility in the Hard Regime). *Under the assumptions of the theorem, exact reliability claims are impossible on the full hard regime.*

(L: [DQ28](#), [DQ38](#).)

Corollary 9.16 (Trilemma of Exact Certification). *In the hard regime, no exact certifier can simultaneously be sound, complete on all in-scope instances, and polynomial-budgeted. Operationally, one of three concessions is unavoidable: abstain on some instances, weaken the guarantee, or risk overclaiming.*

(L: [IC30](#), [IC32](#).)

9.4 Witness and Approximation Limits

Hardness does not disappear when one weakens exact minimization to witness checking or approximation. The first theorem shows that even the empty-set core can force exponential witness inspection. The remaining results then show two complementary approximation obstructions: a direct gap obstruction on the shifted hard family and an exact optimization transfer on the set-cover gadget family.

9.4.1 Witness-Checking Lower Bound

Definition 9.17 (Slot-Inspection Checker). For the empty-set sufficiency core on Boolean state space $S = \{0, 1\}^n$, a *slot-inspection checker* is a deterministic algorithm that adaptively queries witness slots of the form

$$\{(0, z), (1, z)\} \quad z \in \{0, 1\}^{n-1},$$

and, for each queried slot, learns whether the optimizer agrees or disagrees on the two states in that slot. The checker is *sound* if it never accepts a non-sufficient instance and never rejects a sufficient one.

The lower bound below is for deterministic checkers in exactly this access model.

Theorem 9.18 (Witness-Checking Lower Bound). *For Boolean decision problems with n coordinates, any sound slot-inspection checker for the empty-set sufficiency core must inspect at least 2^{n-1} witness pairs in the worst case.*

(L: [WD1-3](#).)

Proof. Let $S = \{0, 1\}^n$. Empty-set sufficiency is exactly the claim that Opt is constant on all states. Partition S into 2^{n-1} disjoint witness slots

$$\{(0, z), (1, z)\} \quad \text{for } z \in \{0, 1\}^{n-1}.$$

Each slot can independently carry a disagreement witness (different optimizer values on its two states).

Consider two instance families:

- *YES instance*: Opt constant on all of S .
- *NO_z instance*: identical to YES except slot z has a disagreement.

If a checker inspects fewer than 2^{n-1} slots, at least one slot z^* is uninspected. On all inspected slots, the YES instance and the tailored NO_{z*} instance are identical. The checker therefore sees exactly the same transcript on these two inputs and must return the same answer on both.

But YES is an empty-set-sufficient instance, whereas NO_{z*} is not. So a common answer cannot be sound on both inputs. Hence any sound worst-case checker must inspect every slot, i.e., at least 2^{n-1} witness pairs. ■

9.4.2 Approximation Gap on the Shifted Family

For the shifted reduction family used in the mechanization, the optimum support size exhibits a sharp gap:

$$\text{OPT}(\varphi) = 1 \quad \text{if } \varphi \text{ is a tautology,} \quad \text{OPT}(\varphi) = n + 1 \quad \text{if } \varphi \text{ is not.}$$

One coordinate acts as a gate: toggling it changes the optimizer even before the formula variables are considered, so optimum size is never 0. If φ is a tautology, every branch behind that gate behaves identically, so all non-gate coordinates become irrelevant and the singleton gate coordinate is sufficient. If φ is not a tautology, choose a falsifying assignment a . For each formula coordinate i , compare the open reference state with the state that inserts a at coordinate i : these states agree on all other coordinates but induce different optimal-action sets, so coordinate i is relevant. Together with gate relevance, this forces every coordinate to be present in every sufficient set. That is what creates the exact 1 versus $n + 1$ gap.

Theorem 9.19 (Approximation-hardness theorem on the shifted family). *Fix $\rho \in \mathbb{N}$. Let $\mathcal{A}$ be any solver that, on every shifted-family instance, returns a sufficient set whose cardinality is within factor ρ of optimum. Then for every formula on n coordinates with $\rho < n + 1$,*

$$\varphi \text{ is a tautology} \iff |\mathcal{A}(\varphi)| \leq \rho.$$

(L: DQ20, DQ39, DQ43.)

Proof. **Tautology $\Rightarrow$ threshold passes.** If φ is a tautology, the optimum support size on the shifted family is 1, realized by the singleton gate coordinate. A factor- ρ solver therefore returns a sufficient set of size at most $\rho \cdot 1 = \rho$, so $|\mathcal{A}(\varphi)| \leq \rho$.

Threshold passes $\Rightarrow$ tautology. Assume $|\mathcal{A}(\varphi)| \leq \rho$ with $\rho < n + 1$. If φ were not a tautology, then every coordinate would be necessary on the shifted family, so every sufficient set would have size exactly $n + 1$. Since $\mathcal{A}(\varphi)$ is sufficient by assumption on $\mathcal{A}$, this would force $|\mathcal{A}(\varphi)| = n + 1 > \rho$, contradiction.

Therefore $|\mathcal{A}(\varphi)| \leq \rho$ holds exactly in the tautology case. ■

Theorem 9.20 (Counted-runtime threshold decider). *Fix $\rho \in \mathbb{N}$. Let $\mathcal{A}$ be any counted polynomial-time factor- ρ solver for the shifted minimum-sufficient-set family. Then the derived procedure*

$$\varphi \mapsto \mathbf{1}\{|\mathcal{A}(\varphi)| \leq \rho\}$$

is a counted polynomial-time tautology decider on the gap regime $\rho < n + 1$.

(L: [DQ23-24](#).)

Proof. Correctness is exactly the gap-threshold theorem above: on the regime $\rho < n + 1$, the predicate $|\mathcal{A}(\varphi)| \leq \rho$ is equivalent to tautology.

For runtime, the derived decider performs two operations: one call to the counted polynomial-time solver $\mathcal{A}$, followed by one cardinality comparison against the fixed threshold ρ . The second step contributes only constant additive overhead relative to the solver run. Hence the derived decision procedure remains counted polynomial-time. ■

9.4.3 Set-Cover Transfer Boundary

Theorem 9.21 (Exact set-cover equivalence on the gadget family). *On the mechanized gadget family, a coordinate set is sufficient if and only if the corresponding set family is a cover. In particular, feasible solutions are in bijection and optimum cardinalities coincide exactly.*

(L: [DQ11](#), [DQ34](#).)

Proof. The gadget is constructed so that each universe element u gives two states, (false, u) and (true, u) , with different optimal-action sets. A coordinate i distinguishes this pair exactly when the corresponding set covers u .

If I is not a cover, choose an uncovered universe element u . Then every coordinate in I takes the same value on (false, u) and (true, u) , but their optimal-action sets differ. So I is not sufficient.

If I is a cover, then for every universe element there exists some coordinate in I that separates the two tagged states attached to that element. Hence two states that agree on all coordinates in I cannot disagree in the tag component in a way that changes the optimizer. The optimizer therefore factors through the I -projection, so I is sufficient.

This yields an exact bijection between feasible sufficient sets and feasible covers, preserving cardinality. Therefore optimum values coincide on the gadget family. ■

Corollary 9.22 (Conditional approximation transfer). *Any factor guarantee or instance-dependent ratio guarantee for minimum sufficiency on the gadget family induces the same guarantee for set cover on that family. Consequently, any impossibility result proved for set cover on that family transfers immediately to minimum sufficiency.*

(L: [DQ25](#), [DQ32-33](#).)

Proof. Compose the candidate minimum-sufficiency solver with the gadget translation and then use the exact equivalence theorem above. Because feasible sets correspond bijectively and preserve cardinality, both approximation factors and instance-dependent ratios are unchanged. ■

The two reductions serve different purposes. The shifted family gives a direct gap obstruction: factor approximation already decides tautology there. The set-cover family gives an exact optimization equivalence: sufficiency is cover there. Together they show that weakening exact minimization to witness checking or approximation does not dissolve the core obstruction.

9.5 Externalized Relevance and Simplicity Tax

The final implication concerns architectural compression. The main point is interpretive rather than algebraic: shrinking a central interface does not remove exact relevance; it relocates the unresolved part of that relevance to some other part of the system. The simplicity tax is the name for that externalized burden.

Definition 9.23 (Relevant Support). For decision problem $\mathcal{D}$, let

$$R(\mathcal{D}) := \{i \in \{1, \dots, n\} : i \text{ is relevant}\}$$

as in Definition 2.8.

Definition 9.24 (Intrinsic and Central Degrees of Freedom). For a model M with native coordinate set $A_M \subseteq \{1, \dots, n\}$:

$$\text{intrinsicDOF}(\mathcal{D}) := |R(\mathcal{D})|, \quad \text{centralDOF}(M, \mathcal{D}) := |R(\mathcal{D}) \cap A_M|.$$

Definition 9.25 (Simplicity Tax).

$$\text{simplicityTax}(M, \mathcal{D}) := |R(\mathcal{D}) \setminus A_M|.$$

The following partition identity makes this precise: any exact simplification that removes relevant coordinates from the central interface must discharge their effect somewhere else in the system.

Proposition 9.26 (Simplicity-Tax Identity). *For every finite-coordinate pair $(M, \mathcal{D})$,*

$$\text{centralDOF}(M, \mathcal{D}) + \text{simplicityTax}(M, \mathcal{D}) = \text{intrinsicDOF}(\mathcal{D}).$$

(L: HD5, DQ9.)

Proof. Immediate from the partition $R(\mathcal{D}) = (R(\mathcal{D}) \cap A_M) \dot{\cup} (R(\mathcal{D}) \setminus A_M)$. ■

In other words, central simplification changes where exact relevance is handled, not how much exact relevance the deployment must ultimately account for.

Proposition 9.27 (Linear Externalization Under a Per-Site Cost Model). *If each unresolved relevant coordinate induces one unit of per-site external work, then for N decision sites:*

$$\text{ExternalWork}(N) = N \cdot \text{simplicityTax}(M, \mathcal{D}).$$

(L: HD26.)

Proof. Each site pays one unit for each unresolved relevant coordinate, so the total cost is the number of sites times the number of unresolved relevant coordinates. ■

Theorem 9.28 (Amortization Threshold). *Let $H_{\text{central}} > 0$ be one-time centralization cost and $\lambda > 0$ the per-site cost coefficient. If $\text{simplicityTax}(M, \mathcal{D}) > 0$, then for*

$$N > \frac{H_{\text{central}}}{\lambda \cdot \text{simplicityTax}(M, \mathcal{D})},$$

the complete model is cheaper than repeated external handling:

$$H_{\text{central}} < \lambda N \cdot \text{simplicityTax}(M, \mathcal{D}).$$

(L: [HD4](#).)

Proof. Repeated external handling costs $\lambda N \cdot \text{simplicityTax}(M, \mathcal{D})$, while the complete model costs H_{central} once. Solving the strict inequality $H_{\text{central}} < \lambda N \cdot \text{simplicityTax}(M, \mathcal{D})$ for N gives the stated threshold. ■

Corollary 9.29 (Hardness Conservation). *Fix a model M and decision problem $\mathcal{D}$. Coordinates in $R(\mathcal{D}) \setminus A_M$ are still decision-relevant for exact behavior preservation. Hence any exact deployment must handle them somewhere: by enlarging the central model, by performing local resolution, by demanding extra queries, or by abstaining on some cases. In the hard exact regime, [Theorem 9.18](#) and [Theorem 9.14](#) imply that this burden cannot be discharged for free by a polynomial-budget exact certifier.*

(L: [DQ9](#), [HD23](#).)

Proof. Take any coordinate $i \in R(\mathcal{D}) \setminus A_M$. Because i is relevant, there exist witness states whose optimal-action sets differ in a way that cannot be ignored in any exact representation. If a deployment neither models i centrally nor resolves its effect elsewhere, then its exact behavior would factor through the smaller interface A_M , making every coordinate outside A_M irrelevant. That contradicts $i \in R(\mathcal{D})$.

So omitted relevant coordinates must be handled somewhere outside the central interface. In the hard exact regime, the witness lower bound shows that exact certification can require exponentially many witness checks, and the integrity-resource theorem shows that polynomial-budget exact competence is unavailable under $P \neq \text{coNP}$. Therefore this external burden cannot, in general, be eliminated for free by a polynomial-budget exact certifier. ■

10 Related Work

10.1 Formalized Complexity Theory and Mechanized Reductions

Machine verification of complexity-theoretic arguments remains substantially less mature than formal verification in algebra, analysis, or standard algorithmics. Forster et al. [\[10\]](#) developed certified machine models and computability infrastructure in Coq, and Kunze et al. [\[19\]](#) formalized major proof-theoretic components in Coq as well. In Isabelle/HOL, much of the mature work has centered on verified algorithms and resource analysis for concrete procedures rather than on families of hardness reductions [\[22, 14\]](#). Lean 4 and Mathlib provide increasingly capable foundations for mechanized finite mathematics and computation [\[32, 8\]](#), but reusable reduction suites for $\text{coNP}/\Sigma_2^P/\text{PP}/\text{PSPACE}$ -style arguments remain relatively rare.

The present artifact is intended as a problem-specific certification layer within that broader program. It does not claim to settle formal complexity theory in full generality; instead, it internalizes the reduction-correctness lemmas, size-bounded hardness packages, exact finite deciders, counted-search procedures, tractability wrappers, explicit-state upper-bound interfaces, and the finite witness/checking machinery now used for the stochastic NPP^{PP} upper-bound story. In that sense the contribution is closest to a certified reduction-and-decision core for one theorem family rather than to a general-purpose complexity library.

10.2 Rough Sets, Reducts, and Attribute Reduction

The closest classical neighbor is rough-set theory and its large literature on reducts and attribute reduction [\[24, 18, 17\]](#). That literature asks when a smaller attribute set preserves discernibility or

decision-table structure, and it has developed both exact and heuristic methods for reduct computation.

10.3 Backdoor Tractability in CSPs

A closely related line of work studies backdoor tractability for constraint satisfaction problems [33, 2]. In that framework, a *backdoor* is a minimal variable set whose fixation collapses a CSP to a tractable class (e.g., one admitting a majority polymorphism). Bessiere et al. [2] showed that testing membership in tractable classes characterized by majority or conservative Maltsev polymorphisms is polynomial-time, but finding a minimal backdoor to such a class is W[2]-hard when the tractable subset is unknown and fixed-parameter tractable only when that subset is given as input. This directly parallels our structural results: coordinate sufficiency in decision problems and backdoor tractability in CSPs both formalize the same intuition—that discovering what matters is fundamentally harder than exploiting a known structure. Our bounded-treewidth reduction (Theorem 8.13) explicitly connects to their CSP algorithms and backdoor methodology.

The complexity of computing minimal reducts has been studied in the rough-set literature. Finding a minimal attribute set that preserves decision-distinctions is known to be NP-hard in general [30], and more refined analysis places certain reduct variants in Σ_2^P [34]. These results establish that the static regime questions studied here, whether a coordinate set preserves decision distinctions, and whether a minimal such set exists, fall within a known complexity landscape for attribute reduction.

The key structural difference is this: general decision tables can admit multiple incomparable minimal reducts, necessitating a combinatorial search through the reduct lattice. Under the exact optimal-action preservation predicate studied here, Proposition 2.9 proves there is exactly one minimal sufficient set: the relevant-coordinate set itself. This uniqueness is the structural reason for the complexity collapse. The search layer vanishes, reducing exact minimization to parallel coNP relevance checks. To our knowledge, this specific structural collapse, from Σ_2^P search to coNP relevance containment driven by optimizer-quotient uniqueness, is a novel classification not captured by general rough-set bounds.

Both settings ask which coordinates can be removed without losing decision distinctions.

Proposition 10.1 (Static Sufficiency as Reduct Preservation for the Induced Decision Table). *Given a finite decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$, form the induced decision table whose condition attributes are the coordinates X_i and whose decision label at state s is the set-valued class label $\text{Opt}(s) \subseteq A$. Then a coordinate set I is sufficient in the sense of this paper if and only if the projection to I preserves the decision label of the induced table. Consequently, the minimal sufficient sets of $\mathcal{D}$ are exactly the reducts of this induced decision table.*

Proof. By definition, I is sufficient exactly when any two states agreeing on I have the same optimal-action set. In the induced decision table, the decision label of state s is precisely $\text{Opt}(s)$, so the same condition says that projection to I preserves the decision label. Minimality on the present side is therefore exactly reduct minimality for the induced table. ■

Rough-set reducts are formulated relative to indiscernibility relations or decision tables, whereas the object here is the optimizer map of a decision problem and the induced exact preservation of optimal-action sets. Proposition 10.1 shows that the static regime can be recast as reduct preservation for the induced decision table. What the present paper adds is: (i) the collapse theorem showing minimum sufficiency collapses to relevance containment, bypassing the general NP-hardness barrier for exact minimization; and (ii) the systematic extension to stochastic and sequential regimes, which

takes the reduct question beyond the classical rough-set setting. The stochastic and sequential layers introduce conditional-comparison and temporal-structure phenomena that are outside the usual rough-set formulation.

The same generosity is due to the broader feature-selection and subset-selection literature in machine learning and combinatorial optimization [4, 1]. Those works study sparse predictive representations, informative feature subsets, and the computational cost of choosing them. Our setting differs in targeting exact preservation of the optimal-action correspondence rather than prediction loss, classifier complexity, or approximate empirical utility. Still, the shared concern is clear: identifying what information matters and what can be safely omitted.

10.4 Decision-Theoretic Sufficiency, Informativeness, and Value of Information

There is also a strong conceptual connection to statistical decision theory and the theory of informativeness. Classical sufficient statistics, value-of-information analysis, and comparison of experiments ask when one information structure is at least as useful for decision making as another [3, 29, 26, 15]. That tradition studies when information can be compressed, compared, or discarded without harming decisions. The optimizer quotient and the exact sufficiency predicate in this paper should be read against that background.

At the same time, the present work studies a different layer of the problem. The focus here is not statistical estimation, asymptotic identifiability, or expected utility under a fixed experiment class. It is the computational certification problem of proving that a coordinate projection preserves the optimal-action correspondence exactly. The connection to classical decision theory is therefore one of ancestry and motivation rather than direct theorem transfer. The paper owes that literature recognition for posing, in older language, the same high-level question: what information is genuinely needed for a decision?

10.5 Abstraction, Bisimulation, and Homomorphisms in Planning and Reinforcement Learning

The stochastic and sequential parts of the paper sit near a second major literature: exact abstraction in Markov decision processes, POMDPs, bisimulation-based aggregation, and MDP homomorphisms [23, 20, 21, 12, 27]. This body of work studies when state spaces can be aggregated while preserving value or policy structure, and it has established many of the representation-sensitive complexity phenomena that make richer stochastic and sequential models computationally difficult.

That literature is structurally close to the present paper and should be credited as such. The main difference is that our predicate is coordinate sufficiency for preserving optimal-action sets under an explicit coordinate-hiding operation. We do not study arbitrary abstract state maps, approximate bisimulation metrics, or value-function approximation guarantees. Instead, we ask for exact certification that a chosen coordinate interface is enough. This exact-certification emphasis is what the paper adds beyond prior abstraction results: the question is not only whether a smaller representation can support good control, but whether one can certify that no omitted coordinate changes the optimal-action correspondence. The contribution here is therefore not abstraction theory in general, but a unified exact-certification account in which static, stochastic, and sequential regimes are analyzed as variants of one decision-relevance question.

10.6 Explanations, Anchors, and Minimal Decision Rules

The paper should also acknowledge adjacent work in explainable AI and local rule-based explanations, especially anchor-style explanations [28]. Readers will notice the word “anchor” and map it

to that literature. The resemblance is real: both settings isolate a restricted condition under which a local piece of information suffices to support a decision or prediction. More broadly, explanation methods based on local rules, counterfactuals, or minimal sufficient subsets share the same informal ambition of identifying a small decision-preserving core.

The difference is again exactness and scope. Anchor explanations in XAI are typically local, model-agnostic, approximate, or precision-calibrated. Our anchor-sufficiency problem is an exact certification problem inside a fully specified decision model, and its complexity is governed by an existential choice followed by a universal fiberwise verification condition. So the paper is not an explanation method, but it is in direct conversation with explanation work that asks what small local structure is enough to preserve a decision.

There is an even closer logical neighbor in recent work on reasons behind decisions, prime-implicant style explanations, and exact decision justifications [7]. This shows that exact small decision-preserving structures are a live contemporary topic, not only a classical one. That literature is concerned with exact structural reasons for a decision outcome and therefore lies much nearer to the present paper than generic post-hoc explanation methods do. The overlap is substantial at the conceptual level: both ask for exact small structures that preserve a decision. The difference is that our objects are coordinate sufficiency, minimum sufficiency, and anchor sufficiency inside coordinate-structured decision problems, and the paper’s main contribution is a regime-sensitive complexity classification of these predicates rather than a logical semantics of reasons alone.

10.7 Abduction, Diagnosis, and Exact Explanatory Cores

The minimum and anchor queries share structural kinship with abduction and diagnosis frameworks, which study explanatory hypotheses sufficient for a target consequence. The anchor query retains an outer existential choice analogous to explanatory discovery, while the inner verification condition preserves optimizer structure. This logical-complexity intuition motivates the Σ_2^P -completeness of static anchor sufficiency, which remains hard even when minimum sufficiency collapses to coNP.

10.8 Causality, Responsibility, and Structural Explanation

The paper also belongs in conversation with structural accounts of causality, explanation, and responsibility [25, 31, 13, 6]. A relevant coordinate is precisely a coordinate whose perturbation can change the decision, and the optimizer quotient isolates exactly the distinctions that matter for optimal-action structure. The paper focuses on exact decision preservation under coordinate projection and on the computational cost of certifying that preservation, whereas causality and responsibility frameworks typically emphasize semantic characterization or intervention-level explanation.

10.9 Oracle, Query, and Access-Based Lower Bounds

Query-access lower bounds provide another nearby technique family [9]. Our witness-checking duality and access-obstruction results belong to that tradition in spirit. The difference is that the lower bounds are developed around the same fixed sufficiency predicate used throughout the paper, which allows direct comparison between forward evaluation, certification, exact minimization, and restricted-access models without changing the underlying decision relation.

10.10 Scope and Novelty

The paper is intentionally synthetic. Rough sets and reducts study decision-preserving attribute reduction; feature selection studies informative subsets; statistical decision theory studies informativeness and sufficiency; abstraction and bisimulation literatures study exact aggregation of stochastic and sequential decision processes; explanation methods study local decision-preserving cores; formalized complexity work studies certified reductions and machine-checked decision procedures. The contribution here is to treat exact relevance certification itself as the common object and to develop one coherent complexity-theoretic account around it.

What is distinctive in the present paper is not an isolated static or sequential hardness theorem taken on its own. The static regime can be recast as reduct preservation for the induced decision table, and the sequential row belongs near existing exact abstraction phenomena in richer control models. The contribution here is to formalize one exact decision-preservation predicate in optimizer language, track it coherently across static, stochastic, and sequential regimes, expose the preservation/decisiveness split in the stochastic case, identify the tractable and intractable boundaries of exact certification, and support the resulting finite combinatorial core with mechanized verification. The paper’s novelty is therefore best understood as a unified exact-certification theory rather than as a single isolated theorem.

11 Conclusion

This paper develops a regime-sensitive theory of exact relevance certification for coordinate-structured decision problems. Its aim is to characterize what information a decision actually depends on, how that information is represented by the optimizer quotient, and when exact certification of that dependence is tractable, split, or fundamentally limited across static, stochastic, and sequential regimes.

Main Results

Within the encoding model of Section 2.4, we establish:

- **Static structure and regime matrix:** in the static regime, the exact information content of the decision is the relevant-coordinate set itself: sufficient sets are exactly its supersets, so exact minimization reduces to relevance containment and is coNP-complete rather than a genuine Σ_2^P search problem, while ANCHOR-SUFFICIENCY remains Σ_2^P -complete. In the stochastic regime, preservation (stochastic sufficiency) has a polynomial-time base query in the explicit-state model with proved bridges to static sufficiency and the optimizer quotient; the minimum and anchor preservation variants are likewise polynomial-time in explicit state, inherit coNP-completeness and Σ_2^P -completeness under full support, and admit support-sensitive obstruction and bridge lemmas beyond full support. The obstruction to general-distribution classification is structural: preservation quantifies over all states, while stochastic analysis is driven by probability-weighted fibers. Zero-probability states can witness violations that the distribution itself never observes, so full-support results do not transfer directly. The support-sensitive obstruction lemmas in the artifact isolate the exact combinatorial mechanism. Decisiveness (stochastic decisiveness) has a polynomial-time base query in the explicit-state model and is PP-hard under succinct encoding; the anchor and minimum decisiveness queries are PP-hard and lie in NP^{PP} , with those oracle-class memberships proved in the paper text and their finite witness/checking cores mechanized. In the sequential regime, sufficiency, minimum, and anchor queries are PSPACE-complete.

- **Encoding-sensitive contrast for exact certification:** logarithmic-size sufficient sets admit polynomial-time algorithms in the explicit regime, while linear-size sufficient sets inherit ETH-conditioned exponential lower bounds in the succinct regime.
- **Structural and downstream consequences:** the paper derives witness and approximation obstructions, exact-simplification hardness transfers, and a direct reliability corollary for exact certifiers in the hard regime: no exact certifier can simultaneously be sound, complete, and polynomial-budgeted on that regime. A simplicity-tax framing shows that omitted exact relevance is externalized rather than eliminated.
- **Structured tractability:** tractable subcases across two tiers: simple edge cases (single action, bounded state space, strict dominance, constant optimal set) admitting immediate checks, and structurally nontrivial cases (bounded actions, separable utility, low tensor rank, tree structure, bounded treewidth, coordinate symmetry) each removing a distinct hardness source with a mechanized reduction or decision procedure.

Taken together, these results show that exact relevance certification is a distinct complexity-theoretic object in its own right: computing $\text{Opt}(s)$ on one state can be easy even when proving coordinate irrelevance is hard, and the difficulty of that certification problem depends sharply on whether one is ruling out counterexamples, preserving optimizers under coarser conditioning, checking decisiveness of conditional fibers, or reasoning about temporally structured optimizer behavior.

Implications

The implications are direct hardness transfers. Configuration simplification is an instance of sufficiency checking, so exact behavior-preserving minimization has no general-purpose polynomial-time worst-case solution unless $\mathbf{P} = \mathbf{coNP}$. The same issue appears in real pipeline orchestration settings, where a nested configuration object controls well selection, output layout, and downstream materialization. There too the practical question is whether one can hide part of the configuration interface without changing the induced compilation or execution decisions. Our results show that this is an exact certification problem whose worst-case difficulty depends on whether one is in the static, stochastic, or sequential regime. Reliable exact certification is likewise regime-limited: under $\mathbf{P} \neq \mathbf{coNP}$, the trilemma of exact certification rules out any solver that is simultaneously sound, complete on the full hard regime, and polynomial-budgeted. And once the canonical relevant-coordinate set $R(\mathcal{D})$ is fixed, any compressed interface merely partitions that relevance into centrally handled and externalized parts. Compression therefore relocates exact relevance rather than erasing it; under the per-site cost model, the unresolved part grows linearly with deployment scale.

This is also the practical meaning of the simplicity tax. In the hard exact regime, a cheap simplification procedure cannot be assumed to have removed decision-relevant structure merely because it has compressed the visible interface. Any relevance omitted from the certified core must still be paid for somewhere else in the system: by local resolution, extra queries, abstention, or weaker guarantees. Exact simplification is therefore costly, but inexact simplification is not free; it externalizes the unresolved burden.

Mechanization

The proof artifact mechanically checks the main reduction constructions, hardness packages, finite deciders, search procedures, tractability statements, step-counting wrappers, the stochastic sufficiency bridge package, the explicit-state decision procedures for the preservation family, the

full-support inheritance results, the new support-sensitive obstruction and bridge lemmas for preservation, the decisiveness upper bounds, and the finite witness/checking schemata for the stochastic existential classification (L: [DC91-99](#), [OV8-12](#), [EH2-10](#).) Full oracle-machine formalization in Lean remains outside the current state of formalized complexity theory; the paper proves the oracle-class memberships in the text following standard conventions, while the artifact provides independent verification of the finite combinatorial core. This places the mechanization in the emerging category of problem-specific certified reduction infrastructure.

Outlook

Two immediate directions remain. First, the reduction infrastructure can be integrated more tightly with general-purpose formalized complexity libraries. Second, the artifact boundary can be made even cleaner by continuing to package the existing finite-decision results into more uniform summary interfaces without changing the paper’s complexity claims. On the complexity side, the remaining gap is now sharply localized: stochastic decisiveness already has succinct hardness and a verified witness/checking core for the oracle-class memberships proved in the paper text, but a fully standard oracle-machine formalization of the resulting oracle-class placement and of the NP^{PP} boundary for the anchor and minimum decisiveness queries remains outside the current repository; on the preservation side, Conjecture 4.1 and Open Problem 4.2 sharpen the remaining open terrain to the full general-distribution and succinct-encoding complexity of the minimum and anchor preservation variants. The mechanized artifact already exposes the finite combinatorial core and support-sensitive lemmas that isolate this obstruction; see `lean_handle_ids_auto.tex` and `claim_mapping_auto.tex` for handles such as [OV12](#), [DC96](#), and [DC98](#), which are appropriate citations when framing the open problem.

Acknowledgments

Generative AI tools (including Codex, Claude Code, Augment, Kilo, and OpenCode) were used throughout this manuscript, across all sections (Abstract, Introduction, theoretical development, proof sketches, applications, conclusion, and appendix) and across all stages from initial drafting to final revision. The tools were used for boilerplate generation, prose and notation refinement, $\text{\LaTeX}$ /structure cleanup, translation of informal proof ideas into candidate formal artifacts (Lean/ $\text{\LaTeX}$), and repeated adversarial reviewer-style critique passes to identify blind spots and clarity gaps.

The author retained full intellectual and editorial control, including problem selection, theorem statements, assumptions, novelty framing, acceptance criteria, and final inclusion/exclusion decisions. No technical claim was accepted solely from AI output. Formal claims reported as machine-verified were admitted only after Lean verification (no `sorry` in cited modules) and direct author review; Lean was used as an integrity gate for responsible AI-assisted research. The author is solely responsible for all statements, citations, and conclusions.

A Lean Handle Index

The table below lists the auto-generated Lean handle identifiers cited inline in the paper.

ID	Handle	ID	Handle
AB2	DecisionProblem.surjective_abstraction_factors_or_erases DecisionQuotient/AbstractionCollapse.lean	APX1	Tractability.CoarseApproximation. finiteUniformErrorRadius_witnesses_uniformApprox DecisionQuotient/Tractability/CoarseApproximation.lean
APX2	Tractability.CoarseApproximation.SampledDocking. SampledDockingProblem.finiteUniformErrorRadius_ witnesses	APX3	Tractability.CoarseApproximation.uniform_approx_ implies_opt_invariance DecisionQuotient/Tractability/CoarseApproximation.lean

ID	Handle	ID	Handle
BP1	Tractability.finiteMinimumGap_le_strictUtilityGap DecisionQuotient/Tractability/CutoffEpsilon.lean	BP2	Tractability.satisfiesBoundedPotential_of_tailBound_and_finiteGap DecisionQuotient/Tractability/CutoffEpsilon.lean
BP3	Tractability.large_cutoff_implies_bounded DecisionQuotient/Tractability/CutoffEpsilon.lean	CB1	Tractability.CoulombApproximation. coulombCutoffErrorRadius_spec DecisionQuotient/Tractability/CoulombApproximation.lean
CB2	Tractability.CoulombApproximation.exact_vs_cutoff_ coulomb_uniformApprox DecisionQuotient/Tractability/CoulombApproximation.lean	CB3	Tractability.CoulombApproximation.exactRealEwald_ satisfiesBoundedPotential_of_tailBound DecisionQuotient/Tractability/CoulombApproximation.lean
CB4	Tractability.CoulombApproximation.exactRealEwald_ satisfiesBoundedPotential_of_tailBound DecisionQuotient/Tractability/CoulombApproximation.lean	CC1	ClaimClosure.DQ8 DecisionQuotient/ClaimClosure.lean
CP1	Tractability.CertifiedPruning.certificate_sound DecisionQuotient/Tractability/CertifiedPruning.lean	CR1	DecisionQuotient.ConfigReduction.config_sufficiency_iff_ behavior_preserving DecisionQuotient/Hardness/ConfigReduction.lean
CT12	DecisionQuotient.Physics.ClaimTransport.physical_bridge_ bundle DecisionQuotient/Physics/ClaimTransport.lean	DC19	StochasticSequential.stochastic_anchor_sufficient_of_ stochastic_sufficient DecisionQuotient/StochasticSequential/Quotient.lean
DC23	StochasticSequential.sequential_anchor_sufficient_of_ sequential_sufficient DecisionQuotient/StochasticSequential/Basic.lean	DC28	StochasticSequential.reduceTQBF_correct_anchor DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC29	StochasticSequential.reduceTQBF_to_sequential_anchor_ reduction DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC40	StochasticSequential.reduceMAJSATPureAnchor_correct DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC41	StochasticSequential.reduceMAJSAT_to_pure_stochastic_ anchor_reduction DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC42	StochasticSequential.stochastic_anchor_check_pp_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC44	StochasticSequential.sequential_anchor_check_pspace_ hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC45	StochasticSequential.stochastic_sufficiency_pp_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC46	StochasticSequential.sequential_sufficiency_pspace_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC47	StochasticSequential.stochastic_minimum_sufficiency_pp_ hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC48	StochasticSequential.sequential_minimum_sufficiency_ pspace_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC49	StochasticSequential.sequentialMinimalSufficient_iff_ relevant DecisionQuotient/StochasticSequential/Basic.lean
DC50	StochasticSequential.sequentialRelevantSet_is_minimal DecisionQuotient/StochasticSequential/Basic.lean	DC57	StochasticSequential.fiberDecisionProblem_sufficient DecisionQuotient/StochasticSequential/Basic.lean
DC62	StochasticSequential.countedStochasticSufficiencySearch_ spec DecisionQuotient/StochasticSequential/Computation.lean	DC63	StochasticSequential.countedStochasticSufficiencySearch_ steps DecisionQuotient/StochasticSequential/Computation.lean
DC64	StochasticSequential.countedStochasticAnchorSearch_spec DecisionQuotient/StochasticSequential/Computation.lean	DC65	StochasticSequential.countedStochasticAnchorSearch_steps DecisionQuotient/StochasticSequential/Computation.lean
DC66	StochasticSequential.countedSequentialSufficiencySearch_ spec DecisionQuotient/StochasticSequential/Computation.lean	DC67	StochasticSequential.countedSequentialSufficiencySearch_ steps DecisionQuotient/StochasticSequential/Computation.lean
DC68	StochasticSequential.countedSequentialAnchorSearch_spec DecisionQuotient/StochasticSequential/Computation.lean	DC69	StochasticSequential.countedSequentialAnchorSearch_steps DecisionQuotient/StochasticSequential/Computation.lean
DC70	DecisionQuotient.static_sufficiency_inP_explicit DecisionQuotient/ExplicitStateMembership.lean	DC71	DecisionQuotient.static_anchor_inP_explicit DecisionQuotient/ExplicitStateMembership.lean
DC72	StochasticSequential.stochastic_sufficiency_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean	DC73	StochasticSequential.stochastic_anchor_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean
DC74	StochasticSequential.sequential_sufficiency_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean	DC75	StochasticSequential.sequential_anchor_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean
DC76	DecisionQuotient.explicit_state_inP_summary DecisionQuotient/ExplicitStateMembership.lean	DC80	DecisionQuotient.staticSufficiency_counted_search_witness DecisionQuotient/AlgorithmComplexity.lean
DC81	DecisionQuotient.static_query_search_matrix DecisionQuotient/AlgorithmComplexity.lean	DC82	DecisionQuotient.finite_search_summary DecisionQuotient/ComplexityMain.lean
DC91	StochasticSequential.countedStochasticPreservationSearch_ spec DecisionQuotient/StochasticSequential/Computation.lean	DC92	StochasticSequential.countedStochasticPreservationSearch_ steps DecisionQuotient/StochasticSequential/Computation.lean
DC93	StochasticSequential.stochastic_preservation_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean	DC94	StochasticSequential.stochastic_preservation_implies_ static_sufficiency DecisionQuotient/StochasticSequential/Basic.lean
DC95	StochasticSequential.static_sufficiency_implies_stochastic_ preservation_of_full_support DecisionQuotient/StochasticSequential/Basic.lean	DC96	StochasticSequential.static_sufficiency_iff_stochastic_ preservation_of_full_support DecisionQuotient/StochasticSequential/Basic.lean
DC97	StochasticSequential.stochasticDecisionEquiv_iff_ decisionEquiv_of_preservation DecisionQuotient/StochasticSequential/Quotient.lean	DC98	StochasticSequential.stochasticDecisionEquiv_iff_ decisionEquiv_of_full_support DecisionQuotient/StochasticSequential/Quotient.lean
DC99	StochasticSequential.stochasticEquivSetoid_eq_ decisionSetoid_of_full_support DecisionQuotient/StochasticSequential/Quotient.lean	DC100	StochasticSequential.benchmark_escalation_summary DecisionQuotient/StochasticSequential/Hierarchy.lean
DN1	DecisionProblem.stateDecisionNoise_iff_same_quotient DecisionQuotient/DecisionNoise.lean	DN2	DecisionProblem.decisionNoise_iff_not_relevant DecisionQuotient/DecisionNoise.lean
DN5	DecisionProblem.decisionNoise_iff_condIndep DecisionQuotient/DecisionNoise.lean	DP1	DecisionQuotient.DecisionProblem.minimalSufficient_iff_ relevant
DP2	DecisionQuotient.DecisionProblem.relevantSet_is_minimal	DP6	ClaimClosure.DP6 DecisionQuotient/ClaimClosure.lean
DP7	ClaimClosure.DP7 DecisionQuotient/ClaimClosure.lean	DQ1	ClaimClosure.DQ1 DecisionQuotient/ClaimClosure.lean
DQ2	ClaimClosure.DQ2 DecisionQuotient/ClaimClosure.lean	DQ3	ClaimClosure.DQ3 DecisionQuotient/ClaimClosure.lean
DQ6	ClaimClosure.DQ6 DecisionQuotient/ClaimClosure.lean	DQ7	ClaimClosure.DQ7 DecisionQuotient/ClaimClosure.lean
DQ9	DecisionQuotient.HardnessDistribution.simplicityTax_ conservation DecisionQuotient/HardnessDistribution.lean	DQ10	DecisionQuotient.IntegrityCompetence.alwaysAbstain_ integrity DecisionQuotient/IntegrityCompetence.lean
DQ11	DecisionQuotient.SetCoverInstance.min_sufficient_iff_set_ cover DecisionQuotient/Hardness/SetCoverReduction.lean	DQ12	DecisionQuotient.StochasticSequential.bounded_horizon_ tractable DecisionQuotient/StochasticSequential/Tractability.lean
DQ13	DecisionQuotient.StochasticSequential.bounded_support_ tractable DecisionQuotient/StochasticSequential/Tractability.lean	DQ14	DecisionQuotient.StochasticSequential.fully_observable_ tractable DecisionQuotient/StochasticSequential/Tractability.lean
DQ15	DecisionQuotient.StochasticSequential.product_ distribution_tractable DecisionQuotient/StochasticSequential/Tractability.lean	DQ16	DecisionQuotient.StochasticSequential.product_enables_ transfer DecisionQuotient/StochasticSequential/CrossRegime.lean
DQ18	DecisionQuotient.StochasticSequential.static_simpler_ than_stochastic DecisionQuotient/StochasticSequential/CrossRegime.lean	DQ19	DecisionQuotient.StochasticSequential.stochastic_simpler_ than_sequential DecisionQuotient/StochasticSequential/CrossRegime.lean

ID	Handle	ID	Handle
DQ20	DecisionQuotient.all_coordinates_necessary_of_not_tautology DecisionQuotient/Hardness/MinSufficientApproximation.lean	DQ25	DecisionQuotient.counted_min_sufficient_inapproximability_conditional DecisionQuotient/Hardness/ApproximationHardness.lean
DQ26	DecisionQuotient.dichotomy_conditional DecisionQuotient/ClaimClosure.lean	DQ27	DecisionQuotient.eth_lower_bound_informal DecisionQuotient/Hardness/ETH.lean
DQ28	DecisionQuotient.exact_certainty_inflation_under_hardness_core DecisionQuotient/ClaimClosure.lean	DQ30	DecisionQuotient.hard_family_all_coords_core DecisionQuotient/ClaimClosure.lean
DQ31	DecisionQuotient.integrity_resource_bound_for_sufficiency DecisionQuotient/ClaimClosure.lean	DQ32	DecisionQuotient.min_sufficient_factor_inapprox_of_set_cover_factor_inapprox DecisionQuotient/Hardness/ApproximationHardness.lean
DQ34	DecisionQuotient.min_sufficient_set_cover_equiv DecisionQuotient/Hardness/ApproximationHardness.lean	DQ36	DecisionQuotient.minsuff_conp_complete_conditional DecisionQuotient/ClaimClosure.lean
DQ38	DecisionQuotient.no_exact_claim_admissible_under_hardness_core DecisionQuotient/ClaimClosure.lean	DQ39	DecisionQuotient.singleton_gate_sufficient_of_tautology DecisionQuotient/Hardness/MinSufficientApproximation.lean
DQ40	DecisionQuotient.sufficiency_conp_complete_conditional DecisionQuotient/ClaimClosure.lean	DQ42	DecisionQuotient.sufficient_means_factorizable DecisionQuotient/Information.lean
DQ43	DecisionQuotient.tautology_decidable_from_factor_approx DecisionQuotient/Hardness/MinSufficientApproximation.lean	DQ44	DecisionQuotient.tractable_subcases_conditional DecisionQuotient/ClaimClosure.lean
DQ45	DecisionQuotient.tractable_tree_core DecisionQuotient/ClaimClosure.lean	DQ51	DecisionQuotient.bounded_actions_tractable DecisionQuotient/Summary.lean
DQ56	DecisionQuotient.orbitType_count_bound DecisionQuotient/Tractability/Dimensional.lean	DQ57	DecisionQuotient.separable_utility_tractable DecisionQuotient/Summary.lean
DQ58	DecisionQuotient.stochastic_objective_bridge_can_fail_on_sufficiency DecisionQuotient/ClaimClosure.lean	DQ62	DecisionQuotient.transition_coupled_bridge_can_fail_on_sufficiency DecisionQuotient/ClaimClosure.lean
DQ63	DecisionQuotient.tree_structure_tractable DecisionQuotient/Summary.lean	DQ72	DecisionQuotient.bounded_actions_complexity DecisionQuotient/Tractability/BoundedActions.lean
DQ74	DecisionQuotient.sufficiency_poly_separable DecisionQuotient/Tractability/SeparableUtility.lean	DQ75	DecisionQuotient.tensor_contraction_tractable DecisionQuotient/Tractability/SeparableUtility.lean
DQ76	DecisionQuotient.low_rank_utility_admits_factored_computation DecisionQuotient/Tractability/SeparableUtility.lean	DQ77	DecisionQuotient.low_rank_tractability DecisionQuotient/Tractability/SeparableUtility.lean
DQ78	DecisionQuotient.sufficiency_poly_tree_structured DecisionQuotient/Tractability/TreeStructure.lean	DQ79	DecisionQuotient.csp_treewidth_tractable DecisionQuotient/Tractability/TreeStructure.lean
DQ80	DecisionQuotient.sufficiency_reduces_to_interaction_csp DecisionQuotient/Tractability/TreeStructure.lean	DQ81	DecisionQuotient.bounded_treewidth_tractability DecisionQuotient/Tractability/TreeStructure.lean
DQ82	DecisionQuotient.orbitType_eq_iff DecisionQuotient/Tractability/Dimensional.lean	DQ83	DecisionQuotient.symmetric_optimalActions_orbit_invariant DecisionQuotient/Tractability/Dimensional.lean
DQ84	DecisionQuotient.sufficiency_reduces_to_cross_orbit_check DecisionQuotient/Tractability/Dimensional.lean	DQ85	DecisionQuotient.symmetric_sufficiency_complexity_bound DecisionQuotient/Tractability/Dimensional.lean
EH1	StochasticSequential.existential_anchor_source_fits_np_over_ppstyle_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean	EH2	StochasticSequential.existential_anchor_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
EH3	StochasticSequential.existential_anchor_query_family_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean	EH4	StochasticSequential.existential_anchor_np_over_ppstyle_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
EH5	StochasticSequential.existential_anchor_query_family_np_over_ppstyle_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean	EH6	StochasticSequential.existential_anchor_query_family_np_over_ppstyle_complete_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
EH7	StochasticSequential.existential_decisiveness_complement_np_over_ppstyle_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean	EH8	StochasticSequential.existential_decisiveness_complement_np_over_ppstyle_complete_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
EH9	StochasticSequential.existential_decisiveness_query_family_np_over_ppstyle_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean	EH10	StochasticSequential.existential_decisiveness_query_family_np_over_ppstyle_complete_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
EX1	Examples.pomdp_reduction_to_preservation DecisionQuotient/Examples/PreservationExamples.lean	EX2	Examples.hyperparam_reduction_to_static
EX3	Examples.toy_full_opt_sl_contains_a DecisionQuotient/Examples/ConcreteExamples.lean	GD1	Tractability.GridMDInstances.gridMDState_sufficient_erase_irrelevant DecisionQuotient/Tractability/GridMDInstances.lean
GD2	Tractability.GridConvergence.resolutionControlledApprox_implies_uniformApprox DecisionQuotient/Tractability/GridConvergence.lean	GD3	Tractability.GridConvergence.lipschitzUtilityApprox_implies_resolutionControlled DecisionQuotient/Tractability/GridConvergence.lean
HD4	DecisionQuotient.HardnessDistribution.complete_model_dominates_after_threshold DecisionQuotient/HardnessDistribution.lean	HD5	DecisionQuotient.HardnessDistribution.gap_conservation_card DecisionQuotient/HardnessDistribution.lean
HD14	DecisionQuotient.HardnessDistribution.linear_lt_exponential_plus_constant_eventually DecisionQuotient/HardnessDistribution.lean	HD23	DecisionQuotient.HardnessDistribution.hardness_is_irreducible_required_work DecisionQuotient/HardnessDistribution.lean
HD26	DecisionQuotient.HardnessDistribution.totalExternalWork_eq_n_mul_gapCard DecisionQuotient/HardnessDistribution.lean	IC30	DecisionQuotient.IntegrityCompetence.exactCertaintyInflation_iff_no_exact_competence DecisionQuotient/IntegrityCompetence.lean
IC32	DecisionQuotient.IntegrityCompetence.integrity_forces_abstention DecisionQuotient/IntegrityCompetence.lean	IC33	DecisionQuotient.IntegrityCompetence.integrity_not_competent_of_nonempty_scope DecisionQuotient/IntegrityCompetence.lean
IC34	DecisionQuotient.IntegrityCompetence.integrity_resource_bound DecisionQuotient/IntegrityCompetence.lean	LJ1	Tractability.LJApproximation.ljCutOffErrorRadius_spec DecisionQuotient/Tractability/LJApproximation.lean
LJ2	Tractability.LJApproximation.exact_vs_cutoff_lj_uniformApprox DecisionQuotient/Tractability/LJApproximation.lean	LJ3	Tractability.LJApproximation.exact_vs_cutoff_lj_opt_invariance DecisionQuotient/Tractability/LJApproximation.lean
LJ4	Tractability.LJApproximation.exactLJ_is_BoundedPotential DecisionQuotient/Tractability/LJApproximation.lean	LJ5	Tractability.SoftLJApproximation.ljSofteningErrorRadius_spec DecisionQuotient/Tractability/SoftLJApproximation.lean
LJ6	Tractability.SoftLJApproximation.exact_vs_softened_lj_uniformApprox DecisionQuotient/Tractability/SoftLJApproximation.lean	LS1	Tractability.LatticeSum.latticeTailSum6_le_M_div_R3 DecisionQuotient/Tractability/LatticeSum.lean
LS2	Tractability.LatticeSum.latticeTailSum12_le_M_div_R9 DecisionQuotient/Tractability/LatticeSum.lean	LS3	Tractability.LatticeSum.lj6_tail_bound DecisionQuotient/Tractability/LatticeSum.lean
LS4	Tractability.LatticeSum.lj12_tail_bound DecisionQuotient/Tractability/LatticeSum.lean	MD1	Tractability.MolecularSrank.outside_cutoff_is_irrelevant DecisionQuotient/Tractability/MolecularSrank.lean
MD2	Tractability.MolecularSrank.md_relevant_only_if_within_cutoff DecisionQuotient/Tractability/MolecularSrank.lean	MD3	Tractability.MolecularSrank.md_srank_bound DecisionQuotient/Tractability/MolecularSrank.lean
MD4	Tractability.MolecularSrank.small_pocket_low_srank DecisionQuotient/Tractability/MolecularSrank.lean	MD5	Tractability.MolecularSrank.docking_small_pocket_bound DecisionQuotient/Tractability/MolecularSrank.lean

ID	Handle	ID	Handle
MD6	Tractability.MolecularSrank.md_srank_matches_dq_definition DecisionQuotient/Tractability/MolecularSrank.lean	MD7	Tractability.MolecularSrank.md_thermodynamic_lower_bound DecisionQuotient/Tractability/MolecularSrank.lean
OU1	StochasticSequential.stochastic_anchor_query_fits_np_over_ppstyle DecisionQuotient/StochasticSequential/OracleUpperBounds.lean	OU2	StochasticSequential.stochastic_minimum_query_fits_np_over_ppstyle DecisionQuotient/StochasticSequential/OracleUpperBounds.lean
OU6	StochasticSequential.stochastic_anchor_inP_explicit_via_witness_schema DecisionQuotient/StochasticSequential/OracleUpperBounds.lean	OU7	StochasticSequential.stochastic_minimum_inP_explicit_via_witness_schema DecisionQuotient/StochasticSequential/OracleUpperBounds.lean
OU8	StochasticSequential.fitsCoNPOverPPStyle_of_no_witness DecisionQuotient/StochasticSequential/OracleUpperBounds.lean	OU9	StochasticSequential.stochastic_decisiveness_query_fits_conp_over_ppstyle DecisionQuotient/StochasticSequential/OracleUpperBounds.lean
OU10	StochasticSequential.stochastic_decisiveness_complement_fits_np_over_ppstyle DecisionQuotient/StochasticSequential/OracleUpperBounds.lean	OU11	StochasticSequential.stochastic_decisiveness_scoped_oracle_bounds DecisionQuotient/StochasticSequential/OracleUpperBounds.lean
OU12	StochasticSequential.stochastic_preservation_explicit_summary DecisionQuotient/StochasticSequential/OracleUpperBounds.lean	PA3	Physics.AnchorChecks.stochastic_anchor_check_iff_exists_anchor_singleton DecisionQuotient/Physics/AnchorChecks.lean
QT1	DecisionProblem.quotient_is_coarsest DecisionQuotient/Quotient.lean	QT7	DecisionProblem.quotient_has_unique_factorization DecisionQuotient/Quotient.lean
SD1	Tractability.SampledDockingGap.strictUtilityGap_lifted_exact DecisionQuotient/Tractability/SampledDockingGap.lean	SD2	Tractability.SampledDockingGap.sampled_epsilon_margin_invariance DecisionQuotient/Tractability/SampledDockingGap.lean
SD3	Tractability.SampledDockingGap.SampledDockingProblem.exact_coarse_opt_agree_of_gap	SD4	Tractability.SampledDockingCutoff.sampled_md_relevant_only_if_within_cutoff DecisionQuotient/Tractability/SampledDockingCutoff.lean
SD5	Tractability.SampledDockingCutoff.sampled_outsideCutoffCoord_irrelevant DecisionQuotient/Tractability/SampledDockingCutoff.lean	SD6	Tractability.SampledDockingCutoff.sampled_md_srank_bound DecisionQuotient/Tractability/SampledDockingCutoff.lean
SD7	Tractability.SampledDockingCutoff.sampled_small_pocket_low_srank DecisionQuotient/Tractability/SampledDockingCutoff.lean	SD8	Tractability.SampledDockingCutoff.sampled_potentialRelevantCoords_sufficient_of_relevance_subset DecisionQuotient/Tractability/SampledDockingCutoff.lean
SD9	Tractability.SampledDockingCutoff.sampled_insideCutoff_sufficient DecisionQuotient/Tractability/SampledDockingCutoff.lean	TK1	Tractability.NearTieBand.exact_topK_subset_ambiguityBand DecisionQuotient/Tractability/NearTieBand.lean
TK2	Tractability.RankingPreservation.pairwise_order_preserved_of_uniform_error DecisionQuotient/Tractability/RankingPreservation.lean	TK3	Tractability.RankingPreservation.coarse_gap_positive_of_exact_gap_margin DecisionQuotient/Tractability/RankingPreservation.lean
TK4	Tractability.TopKPreservation.exact_topK_subset_survivorSet_of_margin DecisionQuotient/Tractability/TopKPreservation.lean	TK5	Tractability.TopKPreservation.topK_preserved_of_boundary_gap DecisionQuotient/Tractability/TopKPreservation.lean
TK6	Tractability.TopKPreservation.exclude_of_exact_below_threshold_margin DecisionQuotient/Tractability/TopKPreservation.lean	WD1	DecisionQuotient.checking_witnessing_duality_budget DecisionQuotient/WitnessCheckingDuality.lean
WD2	DecisionQuotient.no_sound_checker_below_witness_budget DecisionQuotient/WitnessCheckingDuality.lean	WD3	DecisionQuotient.checking_time_ge_witness_budget DecisionQuotient/WitnessCheckingDuality.lean

Mechanized Witness Schemas and Oracle-Class Arguments

The paper uses standard oracle-class language for several stochastic upper bounds, especially the coNP^{PP} -style complement packaging for decisiveness and the NP^{PP} upper bounds for the anchor and minimum decisiveness queries. The corresponding oracle-class memberships are proved in the paper text, following standard complexity-theoretic proof conventions. The proof artifact complements those proofs by mechanizing the finite witness/checking schemata and reduction cores that the oracle arguments require.

Concretely, the mechanized layer certifies ingredients of the following form: a bad fiber witnesses failure of decisiveness; an existentially guessed coordinate set or anchor witness reduces the anchor/minimum queries to the corresponding decisiveness-style verifier; and the bounded explicit-state procedures agree with the abstract predicates they are intended to decide. The paper then translates these certified witness/checking packages into the stated oracle-class membership arguments by the usual guess-and-query reasoning in complexity theory. Thus the artifact provides independent verification of the finite combinatorial core, while the oracle-class interpretation is established in the manuscript.

Computational Chemistry and Docking Support

The same artifact also now exposes the computational-chemistry support needed for a writing-facing docking paper: molecular structural-rank bounds, cutoff irrelevance lemmas, coarse-vs-exact optimizer invariance statements, bounded-potential hypotheses for Lennard–Jones and Coulomb tails, sampled-docking preservation results, and top- k / ranking-stability consequences. These handles are intentionally grouped here so the manuscript can cite them directly when lifting the decision-

quotient framework into a formally verified virtual docking engine. (*L: MD1-7, LJ1-6, CB1-4, BP1-3, APX1-3, SD1-9, TK1-6, GD1-3, CP1, LS1-4.*)

References

- [1] Edoardo Amaldi and Viggo Kann. On the approximability of minimizing nonzero variables or unsatisfied relations in linear systems. *Theoretical Computer Science*, 209(1-2):237–260, 1998.
- [2] Christian Bessiere, Clement Carbonnel, Emmanuel Hebrard, George Katsirelos, and Toby Walsh. Detecting and exploiting subproblem tractability. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 468–474, 2013.
- [3] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953.
- [4] Avrim L. Blum and Pat Langley. Selection of relevant features and examples in machine learning. *Artificial Intelligence*, 97(1-2):245–271, 1997.
- [5] Hans L. Bodlaender. A tourist guide through treewidth. In *Acta Cybernetica*, volume 11, pages 1–21, 1993.
- [6] Hana Chockler and Joseph Y. Halpern. Responsibility and blame: A structural-model approach. *Journal of Artificial Intelligence Research*, 22:93–115, 2004.
- [7] Adnan Darwiche and Auguste Hirth. On the (complete) reasons behind decisions. *Journal of Logic, Language and Information*, 32(1):63–88, 2023.
- [8] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pages 625–635. Springer, 2021.
- [9] Shahar Dobzinski and Jan Vondrák. From query complexity to computational complexity. In *Proceedings of the 44th ACM Symposium on Theory of Computing*, pages 1107–1116, 2012.
- [10] Yannick Forster, Edith Heiter, and Gert Smolka. Verified programming of Turing machines in Coq. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 114–127. ACM, 2019.
- [11] Eugene C. Freuder. Complexity of k-tree structured constraint satisfaction problems. *Proceedings of AAAI*, pages 4–9, 1990.
- [12] Robert Givan, Thomas Dean, and Matthew Greig. Equivalence notions and model minimization in markov decision processes. *Artificial Intelligence*, 147(1-2):163–223, 2003.
- [13] Joseph Y. Halpern and Judea Pearl. Causes and explanations: A structural-model approach. part ii: Explanations. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence*, pages 27–34, 2001.
- [14] Maximilian P. L. Haslbeck and Tobias Nipkow. Verified analysis of list update algorithms. In *Journal of Automated Reasoning*, volume 65, pages 117–156. Springer, 2021.
- [15] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.

- [16] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k -sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.
- [17] Richard Jensen. Rough set-based feature selection: A review. In *Rough Computing: Theories, Technologies and Applications*, pages 91–129. IGI Global, 2008.
- [18] Richard Jensen and Qiang Shen. Semantics-preserving dimensionality reduction: Rough and fuzzy-rough-based approaches. *IEEE Transactions on Knowledge and Data Engineering*, 16(12):1457–1471, 2004.
- [19] Fabian Kunze, Gert Smolka, and Yannick Forster. Formal small-step verification of a call-by-value lambda calculus machine. In *Asian Symposium on Programming Languages and Systems*, pages 264–283. Springer, 2019.
- [20] Michael L. Littman, Judy Goldsmith, and Martin Mundhenk. The computational complexity of probabilistic planning. *Journal of Artificial Intelligence Research*, 9:1–36, 1998.
- [21] Martin Mundhenk, Judy Goldsmith, Christopher Lusena, and Eric Allender. The complexity of finite-horizon markov decision process problems. *Journal of the ACM*, 47(4):681–720, 2000.
- [22] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2002.
- [23] Christos H. Papadimitriou and John N. Tsitsiklis. The complexity of markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987.
- [24] Zdzisław Pawlak. Rough sets. *International Journal of Parallel Programming*, 11(5):341–356, 1982.
- [25] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.
- [26] Howard Raiffa and Robert Schlaifer. *Applied Statistical Decision Theory*. Harvard University Press, 1961.
- [27] Balaraman Ravindran. *An Algebraic Approach to Abstraction in Reinforcement Learning*. PhD thesis, University of Massachusetts Amherst, 2004.
- [28] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Anchors: High-precision model-agnostic explanations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [29] Leonard J. Savage. *The Foundations of Statistics*. John Wiley & Sons, 1954.
- [30] Roman Slowinski and Daniel Vanderpooten. Decision under uncertainty: A rough set approach. *European Journal of Operational Research*, 80(2):277–289, 1995.
- [31] Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. MIT Press, 2nd edition, 2000.
- [32] The mathlib Community. The Lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.

- [33] Ryan Williams, Carla P. Gomes, and Bart Selman. Backdoors to typical case complexity. In *Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1173–1178. Morgan Kaufmann Publishers Inc., 2003.
- [34] Xiaofeng Zhang et al. On the complexity of attribute reduction in decision tables. *Fundamenta Informaticae*, 123(4):421–434, 2013.